

Broadcast for Dynamic Committees without Trusted Setup

Gabriel Dettling¹, Chen-Da Liu-Zhang¹, Elisaweta Masserova², Matthieu Rambaud³, and Antoine Urban⁴

¹ Lucerne University of Applied Sciences and Arts

² Carnegie Mellon University

³ Télécom Paris

⁴ DFNS

Abstract. A significant number of works have considered the problem of multi-party computation over dynamic committees in synchronous networks, including YOSO MPC [Crypto’21], Fluid MPC [Crypto’21], SCALES MPC [TCC’22] and Layered MPC [Crypto’23]. However, prior works assume that every party has access to an ideal synchronous broadcast channel towards the next committee.

While this assumption is partly justified due to the seminal work of Garay [WDAG’94] stating that deterministic broadcast with dynamic committees is impossible, it is open whether there are randomized solutions.

We answer this question in the affirmative, by providing a complete characterization of broadcast with dynamic committees. We use the formalization introduced in the Layered MPC setting and achieve the following results for layered broadcast:

- A **statistically secure** protocol tolerating $t < n/3$ corruptions with no setup.
- A **computationally secure** protocol tolerating $t < n/2$ corruptions, assuming only a plain PKI for signatures.
- A matching **impossibility result** showing that broadcast is impossible for $t \geq n/2$ corruptions.

Using our broadcast, we achieve the following polynomial-time results:

- The first YOSO MPC protocols without broadcast (statistical for $t < n/3$ without setup; and computational for $t < n/2$ assuming a plain PKI for signatures).
- Assuming plain PKIs for signatures and public-key encryption, the first Layered MPC protocol without broadcast for $t < n/2$.
- Assuming homomorphic commitments, the first Layered MPC as long as $(t + 1)^2 \leq n$. To achieve this, we introduce a secure-message-transmission protocol for $(t + 1)^2 \leq n$ which has linear communication in ℓ and polynomial communication in n when transmitting a message across ℓ layers. This result is of independent interest.

1 Introduction

Multi-party computation (MPC) [56, 37, 5, 20, 53] allows a set of mutually distrusting parties to perform a joint computation over their private inputs in

such a way that nothing about their data beyond the output of the computation is leaked. Most of the MPC protocols proposed in the literature focused on the traditional setting where there is a fixed set of participants that stay online throughout the computation, and security is guaranteed as long as up to a threshold of the parties is compromised. However, this static model is not well suited for capturing modern large-scale applications that are long-lived (such as blockchains or training federated learning models), and operate in environments where parties may join and leave the computation.

To address these limitations, several models for MPC with dynamic committees have been proposed in recent years, including the You-Only-Speak-Once model (YOSO) MPC [36], Fluid MPC [22], Layered MPC [24] and SCALES [3] models. At a high level, the computation is structured as a sequence of rounds, where in each round there is a different set of active parties, usually called a committee, that sends messages towards future committees. Security is then guaranteed as long as up to a threshold of the parties is compromised in each committee.

Protocols designed in these models successfully capture the dynamism that appears in the setting of MPC with dynamic committees. Unfortunately, to the best of our knowledge, all known protocols assume an ideal broadcast channel: whenever a party in one committee sends a message, it is reliably and consistently delivered to the next committee (and often implicitly to all future committees⁵). Thus, investigating whether it is possible to achieve broadcast itself in the setting of dynamic committees is an important question.

This question was previously studied in the consensus and blockchain literature [21, 11, 33, 49, 1, 41, 51] under so-called player-replaceable protocols⁶. Unfortunately, all player-replaceable broadcast protocols assume a trusted setup (typically in the form of a threshold signature setup, threshold verifiable random functions or sharings of random values to generate unpredictable common randomness), which is a very strong assumption since realizing a trusted setup in an environment where the set of parties is dynamic is challenging. This motivates the central question of our work:

Is it possible to achieve broadcast assuming point-to-point channels in the setting of dynamic committees without trusted setup?

Before diving into the contributions, it is worth noting that the seminal work of Garay [34] showed that a deterministic broadcast protocol is impossible in the setting of dynamic committees even assuming that there is only one single cor-

⁵ This is without loss of generality assuming honest majority per committee, since one can always broadcast to the next committee and further propagate the message towards future committees by forwarding and taking majority decisions.

⁶ The original motivation of player-replaceability and YOSO was to avoid strong adaptive DDoS attacks. The idea was that if a secretly-chosen committee only speaks once, even if the adversary corrupts the committee after it already spoke, this does not harm the protocol since the committee already did their job.

ruption in each committee (see also [24]). This partly justified the assumption of a broadcast channel in prior works, hinting that it may be a necessary assumption. As we show in this work, however, this impossibility can be circumvented by the use of randomization.

1.1 Our Contributions

In this work, we give a complete characterization of broadcast protocols for dynamic committees. We adopt the formalism of Layered MPC [24, 26] (which is equivalent to Fluid MPC with maximal-fluidity [22]) because it cleanly models the dynamic committee setting, but our protocols work under other formalisms for MPC with dynamic committees such as YOSO MPC as well.

In Layered MPC parties are placed in a communication network that has the shape of a layered graph, with n parties in each layer (representing one committee). A party in any layer has access to point-to-point secure channels that allow them to securely communicate to each party in the immediate next layer. Contrary to the original formulation, we do not assume broadcast channels. This means that parties in any layer simply receive private messages from the parties in the previous layer, and send private messages to the parties in the next layer. The adversary is restricted to corrupting a subset of up to t out of n parties in each layer, a specialization of the notion of general adversary structure [42].

We consider the problem of realizing broadcast over a layered network that allows a sender in $\mathcal{L}_0$ to consistently send a message towards all parties belonging to a final layer $\mathcal{L}_d$.

Result 1. We introduce the first broadcast protocol for $t < n/3$ in the Layered MPC setting without any setup and while achieving statistical security.

Theorem 1. *There exists a statistical layered broadcast protocol for $t < n/3$ with polynomial communication.*

Result 2. Since broadcast for $t \geq n/3$ requires some form of setup [46], we next explore the setting where parties have access to plain PKI for signatures⁷.

Theorem 2. *Assuming a plain PKI for signatures, there exists a layered broadcast protocol for $t < n/2$ with polynomial communication.*

Result 3. We then obtain a matching result by observing that layered broadcast is impossible for $t \geq n/2$. This is in contrast to the setting with a fixed number of parties, where broadcast is possible for any $t < n$ [28].

Theorem 3. *There is no layered broadcast protocol for $t \geq n/2$.*

⁷ In a plain PKI the keys from corrupted parties can be chosen adversarially. See [13] for a nice discussion.

Our broadcast protocols can be plugged into existing MPC protocols with dynamic committees. Plugging our broadcast protocols into existing YOSO MPC protocols [36] leads to the first YOSO MPC protocols without broadcast and with guaranteed output delivery.

Corollary 1. *There are YOSO MPC polynomial protocols with guaranteed output delivery without assuming broadcast for 1) $t < n/3$ corruptions with statistical security and no setup, and 2) $t < n/2$ corruptions assuming a plain PKI.*

Our broadcast protocols can also be plugged into existing protocols in the Fluid and Layered MPC settings [24, 26]. A subtlety, however, is that plugging in a broadcast protocol introduces a number of *new intermediate layers* equal to the round complexity of broadcast. Since in Fluid and Layered MPC each committee $\mathcal{L}_k$ only has access to channels towards the immediate next committee $\mathcal{L}_{k+1}$, and our broadcast protocols require a number of rounds proportional to the security parameter λ , any secret message that was being transmitted from $\mathcal{L}_k$ to $\mathcal{L}_{k+1}$ now needs to be transmitted from $\mathcal{L}_k$ to $\mathcal{L}_{k+\Theta(\lambda)}$. This is not a problem in the YOSO MPC model (since channels to future committees are available), or if we assume a plain PKI for public-key encryption (since transmitting a message towards a specific party in a future committee can be done simply by encrypting under the receiver’s public key, and forwarding the message until it reaches the receiver). This observation leads to the following theorem.

Corollary 2. *Assuming plain PKIs for signatures and public-key encryption, there is a maximally Fluid / Layered MPC polynomial protocol with guaranteed output delivery without assuming broadcast for $t < n/2$ corruptions.*

However, without public-key encryption, all known protocols for secure message transmission (SMT) from point-to-point secure channels in the maximally Fluid and Layered MPC settings have a communication that is exponential in both the number of committees and the committee size when transmitting messages across $\Theta(\lambda)$ layers (we need this because our broadcast incurs $\Theta(\lambda)$ rounds).

To mitigate this, we introduce the first SMT protocol that incurs a communication complexity that increases only linearly with the number of committees (its communication complexity is still exponential in the committee size).

Theorem 4. *Assuming non-interactive homomorphic commitments, there is an SMT protocol that transmits a message across ℓ committees with communication $O(\ell \cdot \binom{n}{t+1} \cdot n^3)$ that is secure up to $t < n/2$ corruptions.*

Based on this protocol, which we denote **SMT**, we introduce a variation of it, called **SMT***, which works for $(t+1)^2 \leq n$ and only incurs polynomial communication. To the best of our knowledge, all layered SMT protocols that can transmit a message $\Theta(\lambda)$ layers incur exponential communication even when considering only fail-stop corruptions and a constant number of corruptions.

Corollary 3. *Assuming non-interactive homomorphic commitments, there is a secure message transmission protocol that transmits a message across ℓ committees with polynomial communication $O(\ell \cdot n^{\frac{3}{2}})$ that is secure up to $t < \sqrt{n}$ corruptions.*

By integrating our secure message transmission protocol and our statistical broadcast protocol into the layered MPC protocol in [24], we obtain the first Layered MPC protocol without assuming setup and without a broadcast channel.

Corollary 4. *Assuming non-interactive homomorphic commitments, there is a maximally Fluid / Layered MPC polynomial protocol with guaranteed output delivery without assuming broadcast secure up to $t < \sqrt{n}$ corruptions.*

1.2 Related Work

Several works use the idea of designing player-replaceable protocols that operate with dynamically chosen committees [21, 11, 40, 39] as a way to achieve adaptive security and/or improve the communication pattern. Protocols for consensus and broadcast with player-replaceability have been proposed [21, 11, 33, 49, 1, 41, 51] in the literature, but to the best of our knowledge, all of them require a trusted setup, typically in the form of a unique threshold signatures, verifiable random functions or a sequence of Shamir sharings of random values, to generate unpredictable randomness.

Since then, a broad spectrum of models [36, 22, 24, 54, 3, 31] have been proposed to capture the notion of *multi-party computation with dynamic committees*. We summarize the models and their recent developments in the area of MPC with dynamic committees. To the best of our knowledge, all the protocols introduced in these models up to date assume ideal broadcast channels.

Fluid MPC [22] and Layered MPC [24]. Fluid MPC considers a scenario where the parties in a committee can only send private messages to the next committee, and where there are up to t corruptions in each committee. Each party is required to participate in a certain minimum number of consecutive committees, a parameter referred to as the protocol’s *fluidity*. Layered MPC is as a bare-bones version of the *maximally fluid MPC*⁸, which ignores some of the additional features of the Fluid MPC framework (such as how to sample the committees from the overall set of parties). The original Fluid MPC [22] introduced a maximally Fluid statistical protocol for security with abort under honest majority. Bienstock et al. improved the communication complexity of maximally Fluid protocols to linear in the statistical [9] and perfect [10] settings. Rachuri and Scholl [54] obtained protocols in the dishonest-majority setting. For guaranteed output delivery, David et al. [22] and Deligios et al. [26] obtained perfect and statistical Layered MPC with $t < n/3$ and $t < n/2$, respectively.

⁸ In maximally fluid MPC a party may exit the computation after a single round of receiving and sending messages.

YOSO MPC [36]. The YOSO MPC model considers committees that speak only once, and an idealized communication channel called the role assignment functionality, which ensures that every party in a committee can send private messages to any recipient in an arbitrary future round. In contrast to layered MPC, a dishonest sender cannot change the message while it is traveling to the recipient and is, effectively, committed to its input.⁹ Recent works provided improvements to YOSO MPC across various directions. Braun et al. [14] designed distributed key generation for linearly homomorphic threshold encryption in the YOSO setting. Escudero et al. [30] showed how to obtain YOSO protocols in the offline-online model with (amortized) constant online communication, and Kolby et al. [45] introduced constant-round YOSO MPC protocols. Recently, Bhadauria et al. [8] designed FHE-based YOSO MPC which uses only one honest majority committee. Finally, a number of works focused on obtaining efficient role-assignment protocols [6, 19, 18, 15, 23].

SCALES MPC [3]. SCALES can be seen as a variant of YOSO MPC, where the committee of input and output clients is the same, and, in contrast to YOSO, these clients are allowed to speak twice. Moreover, SCALES requires that the communication is done purely via broadcast (no secure channels available), as well as the complexity of each input client only depends on the size of their inputs and the number of participating computing parties (in particular, it does not depend on the size of the computation). While the original work on SCALES [3] focused on the semi-honest setting, Acharaya et al. [2] provided protocols secure against fully malicious adversaries. Most recently, Acharaya et al. [4] improved efficiency of SCALES for permutation branching programs.

PASSO [50, 47]. A variant of YOSO, called *Parties Speak Sequentially Once* was recently considered in [50, 48, 47] for the concrete problem of randomness generation. This variant considers a setting where committees are executed in sequence, and the adversary can corrupt a total of t committees. The works consider the setting where each committee consists of one single party, and parties have access to channels to the future.

2 Technical Overview

We summarize the technical challenges and the ideas that we used to achieve our results in both the design of the layered broadcast protocols, as well as their integration to achieve layered MPC.

2.1 Layered Broadcast

We focus on achieving a layered version of consensus, where every party $P_i \in \mathcal{L}_0$ has an input x_i and parties in layer $\mathcal{L}_d$ want to agree on the same bit. Moreover,

⁹ The original YOSO work also considered random corruptions instead of worst-case corruptions, but this assumption is not crucial for their protocols.

if all honest parties in $\mathcal{L}_0$ input the same bit b , every honest party in $\mathcal{L}_d$ should output b . Note that once consensus is achieved, one can achieve broadcast simply by letting the sender send his input x towards all parties in $\mathcal{L}_1$, and then running consensus. See appendix C for details.

Background: Feldman-Micali Paradigm. Our starting point is the seminal Feldman-Micali (FM) paradigm [32] for building probabilistic consensus. At a high level, protocols in this paradigm are organized as a sequence of iterations, where each iteration consists of an invocation of two primitives: *weak consensus* and a *weak common coin*.

A weak common coin is a protocol that, with constant probability p , outputs a uniform common random bit $c \in \{0, 1\}$ to all parties. In weak consensus (WC), each party P_i inputs a bit $x_i \in \{0, 1\}$ and outputs a value $z_i \in \{0, 1, \perp\}$, with the guarantee that there is a value z such that each honest party P_i outputs either z or $\perp$, i.e., honest parties do not output contradicting bits. Moreover, if each honest P_i inputs the same bit b , they all output b .

To achieve consensus, each honest party P_i in every iteration performs two steps: 1. it invokes $z_i \leftarrow \text{WC}(x_i)$, and 2. it then runs the common coin protocol to obtain c_i , setting

$$y_i = \begin{cases} z_i, & \text{if } z_i \in \{0, 1\}, \\ c_i, & \text{if } z_i = \perp. \end{cases}$$

Note that after the invocation of WC, honest values are within the interval $\{b, \perp\}$ for some bit $b \in \{0, 1\}$. This means that if the coin is successful (outputs a uniform common random bit to all parties) and equal to b , parties reach agreement. This happens with probability $p/2$.

By repeating the iteration steps $O(\lambda)$ times for security parameter λ , where the input to a new iteration is set to be the output of the previous iteration, agreement is reached with overwhelming probability.

In order to adapt this approach to achieve layered consensus, the main technical challenge is in designing the weak common coin protocol (this is the place where prior work assumes trusted setup), although there are other subtleties that need to be solved along the way. For example, in each iteration, after running WC, the layer holding the output of weak consensus, say $\mathcal{L}_{d_1}$, needs to securely transmit the output to the layer that will hold the output of the common coin $\mathcal{L}_{d_1+d_2}$ (which takes, say, d_2 rounds); see Section 7 for a mechanism to solve this.

Layered Weak Common Coin. At a high level, we adapt the leader election protocol from Katz and Koo [44] to achieve a layered weak common coin. This protocol uses an n -party primitive called *moderated VSS*, which involves two designated parties, the dealer and the moderator. Moderated VSS guarantees that either the dealer successfully shares a secret, or – if the moderator is corrupt – it may fail, but such a failure is guaranteed to be detected by all parties. Note that “false negatives” are possible: a corrupt moderator may select a subset of

honest parties and make them think that the protocol failed. Using n^2 instances of moderated VSS, one for each possible choice of dealer (sharing a random value) and moderator, the protocol then assigns a random *ticket* value to each moderator P_m , namely the sum of all values shared in an instance of moderated VSS where P_m acted as moderator. Then, the secrets are reconstructed, and every party P selects the moderator with the lowest ticket value as their leader, discarding all moderators which P detected as having acted incorrectly.

This works because, although parties may disagree about which moderators should be discarded, it is guaranteed that every moderator considered acceptable by some honest party got assigned a uniformly random ticket value. Thus, there is a reasonably high probability of at least $\frac{n-t}{n} - \frac{1}{n^2}$ that the unique lowest value is assigned to an honest moderator, in which case all honest parties choose the same leader.

Converting the Leader Election to a Coin. To obtain a random coin rather than an oblivious leader election protocol, we duplicate this process, assigning two values to every moderator in the same fashion as described above. The first value (ticket value) is used for choosing the leader as before, while the second value assigned to the leader is used as the coin value. The idea is that while the adversary may influence the choice of leader (by corrupting and disqualifying selected parties), the coin value output by any honest party is uniform random.

Achieving a weak common coin in the layered setting presents a number of challenges.

Challenge 1: Stateless Moderator. First, in the layered setting, it is not possible for one party to act as a moderator for the entire sharing protocol, since prior moderated VSS protocols require the moderator to keep a state throughout multiple rounds (in prior constructions the moderator is in charge of emulating a broadcast channel throughout the protocol execution). To address this, instead of relying on a single stateless moderator that can only speak once in a layer, we introduce a new abstraction *layered moderated VSS* which extends the prior definition to consider *multiple* moderators. More concretely, there is a sequence of moderators $P_{m_1}, \dots, P_{m_k}$ belonging to different layers, and the primitive guarantees that the dealer successfully shares a secret only in the case where all the moderators are honest. Our construction of layered moderated VSS generalizes the technique from [44], introducing a compiler from layered VSS (which use a broadcast channel) to layered moderated VSS (without a broadcast channel). See section 5 for more details.

Note that in the coin protocol, this has the effect of exponentially decreasing the fraction of fully honest moderator sequences $\left(\frac{n-t}{n}\right)^k$ with the number of moderators k . This turns out not to be an issue, as our construction only requires a constant number of moderators.

Challenge 2: Propagating the Layered Moderated-VSS Outcome. In the standard setting with a static set of parties, the parties keep a state throughout the protocol on whether any given moderator is considered honest or not. In our setting, this information needs to be further propagated throughout future layers. We achieve this by invoking one weak consensus protocol for each modera-

tor sequence, and each party only takes into account moderator sequences where the weak consensus outputs 1 (and we have a method to securely propagate weak consensus outputs to future layers). This guarantees that an honest moderator sequence is considered honest throughout future layers; and any moderator sequence with a failed moderated VSS instance is always considered disqualified.

However, this introduces a new vulnerability: a rushing adversary is able to make one honest party P_{h_1} view all corrupt moderators as disqualified after the Sharing phase of the moderated-VSS instances, while another honest party P_{h_2} views all of them as successful. Note that in this case, all moderated-VSS instances are secure (since P_{h_2} considers them successful). During the Reconstruction phase, the adversary can first see the reconstructed secret values, and based on this, influence the outputs of the weak consensus protocols for the corrupted moderators. If a corrupted moderator has the lowest ticket value, the adversary can choose to adaptively disqualify this moderator during the weak consensus. As a consequence, the output of the coin is biased.

Nevertheless, we show that the protocol still achieves a weaker definition of a common coin, namely that every output value occurs with constant positive probability, which suffices for our overall construction.

2.2 Achieving Layered MPC without Broadcast

As mentioned in the introduction, our broadcast protocols can be plugged into existing MPC protocols with dynamic committees. However, plugging a broadcast protocol introduces a number of intermediate layers equal to the round complexity of broadcast, which is $\Theta(\lambda)$. This is not a problem in the YOSO MPC model (since channels to future committees are available), or if we assume a plain PKI for public-key encryption (since transmitting a message towards a specific party in a future committee can be done simply by encrypting under the receiver’s public key, and forwarding the message until it reaches the receiver). However, without public-key encryption, all known protocols for secure message transmission from point-to-point secure channels in the Fluid and Layered MPC settings have a communication that increases exponentially with the number of committees d from sender to receiver, which means that the resulting protocol has exponential communication. We address this by introducing a new secure message transmission protocol which has communication complexity is polynomial when $(t + 1)^2 \leq n$, using homomorphic commitments. This protocol is of independent interest.

Background: Strawman Approach. To illustrate why prior approaches require exponential communication, let us recap a standard approach for $t < n/3$ corruptions. Consider a sender $P_{\text{sdr}} \in \mathcal{L}_0$ with input s and a receiver $P_{\text{rcv}} \in \mathcal{L}_d$. In the case where $d = 1$ (sender and receiver are in subsequent layers), they can directly communicate via the provided secure channel. If $d = 2$, a simple solution consists of sender distributing a t -out-of- n Shamir secret sharing of s to layer $\mathcal{L}_1$, so that each party in this layer holds a share. Each $P_i \in \mathcal{L}_1$ then forwards their share to $P_{\text{rcv}} \in \mathcal{L}_2$, who robustly reconstructs the secret. The general case ($d > 2$) can be done recursively. Assume an SMT protocol Π for distance

$d' = d - 1$. The sender can again distribute a Shamir sharing of the secret towards layer $\mathcal{L}_1$. Each of the shares is then forwarded by parties in layer $\mathcal{L}_1$ to the receiver in $\mathcal{L}_d$ using a separate instance of Π . Note, however, that the communication complexity of this protocol is $O(n^{d-1})$, since number of values received and communicated by each party increases exponentially with the number of intermediate layers d . To the best of our knowledge, all layered SMT protocols suffer from a problem of this kind and incur exponential communication.

Challenge 3: Difficulty in Reducing the State Size. One may wonder whether it is possible to reduce the state size in the previous approach. Indeed, after the sender s distributes Shamir shares to $\mathcal{L}_1$, one could attempt to invoke a proactive secret sharing scheme so that parties in $\mathcal{L}_2$ hold a re-randomized Shamir sharing of the same secret. For example, by letting every $P_i \in \mathcal{L}_1$ hand off their share s_i into sub-shares $(s_{i,1}, \dots, s_{i,n})$ and sending $s_{i,j}$ to $P_j \in \mathcal{L}_2$, who can re-combine the shares $(s_{1,j}, \dots, s_{n,j})$ into a single share s'_j using Lagrange interpolation and linearity of Shamir sharing, so that the new shares $(s'_1, \dots, s'_n)$ constitute a valid degree- t Shamir sharing of the original secret s . Unfortunately, for this (well-known [27, 35]) approach to work, it is crucial that all honest parties in $\mathcal{L}_2$ agree on a consistent set $S \subset \mathcal{L}_1$ of parties that correctly shared their share. At a high level, agreeing on such a set requires broadcast. Note that using our broadcast protocol would again introduce $\Theta(\lambda)$ additional intermediate layers between $\mathcal{L}_1$ and $\mathcal{L}_2$, and the protocol also requires securely sending private sub-shares from $\mathcal{L}_1$ to $\mathcal{L}_2$. Therefore, this becomes a circular problem.

Layered Secure Message Transmission. To securely transmit messages across intermediate layers without using PKI or broadcast channels, we introduce a novel proactive secret sharing scheme based on non-interactive linearly-homomorphic commitments. At a high level, the sender will distribute a commitment C of the secret input to all parties (this commitment can be propagated for $t < n/2$ across all future layers by letting parties forward it to all parties and by taking a majority decision). Parties in each layer will hold multiple Shamir-sharings of the secret with consistent commitments to shares as described below.

Valid Sharing. Given commitment C , a *valid share* of P_i is a tuple $(\ell, \mathbf{C}_i^\ell, s_i^\ell, r_i^\ell)$, with $\mathbf{C}_i^\ell = (C_1^\ell, \dots, C_n^\ell)$, such that 1) $\text{Com}(s_i^\ell, r_i^\ell) = C_i^\ell$, and 2) the points $(0, C)$ and $\{(k, C_k^\ell)\}_{k \in [n]}$ lie on a degree- t polynomial.

We call value ℓ the label of this share. It acts as the **unique** identifier of the share $(\ell, \mathbf{C}_i^\ell, s_i^\ell, r_i^\ell)$ among the shares of party P_i , i.e. each P_i has at most one tuple with each label ℓ . Looking ahead, the label will encode the set of parties of prior layer whose shares were used in the construction of this one.

We then say that parties in layer $\mathcal{L}$ jointly hold a *valid sharing* for label ℓ iff each honest party holds a valid share with label ℓ , and the vector of commitments contained in each such share is the same for all honest parties.¹⁰

Our protocol ensures that, if the dealer is honest, in each layer $\mathcal{L}$ parties jointly hold a valid sharing for at least one label. As mentioned above, honest parties are always in agreement on the commitment of the secret C . However,

¹⁰ Note that there may be valid shares that do not correspond to any joint valid sharing.

parties will not know *which* label corresponds to a joint valid sharing. The fact that they never have to explicitly agree on a right label is what allows us to circumvent the circular problem mentioned above.

Handoff Protocol. We describe our handoff protocol (Handoff, in Figure 1):

Each party P_i in the old layer $\mathcal{L}_{\text{old}}$ has as input (1) the commitment to the secret C , and, (2) a set of valid shares $(\ell, \mathbf{C}_i^\ell, s_i^\ell, r_i^\ell)$ for $\ell \in \mathcal{Q}$, where $\mathcal{Q} := \binom{[n]}{t+1}$. It forwards the commitment to the secret C to each party of the next layer. Additionally, it hands off each of the valid shares $(\ell, \mathbf{C}_i^\ell, s_i^\ell, r_i^\ell)$ as follows. First, it computes Shamir shares of s_i^ℓ, r_i^ℓ , denoted by $(s_{i,1}^\ell, \dots, s_{i,n}^\ell)$ and $(r_{i,1}^\ell, \dots, r_{i,n}^\ell)$ respectively. Then, it computes the vector of commitments to these subshares $\tilde{\mathbf{C}}_i^\ell[j] = \text{Com}(s_{i,j}^\ell, r_{i,j}^\ell)$. Finally, for each $j \in [n]$ it sends $(\ell, \tilde{\mathbf{C}}_i^\ell, s_{i,j}^\ell, r_{i,j}^\ell)$ to party P_j of the next layer $\mathcal{L}_{\text{new}}$.

Each party P_j of the new layer $\mathcal{L}_{\text{new}}$ sets the commitment to the secret to be the commitment value sent by the majority of parties in the old layer. Denote this commitment (again) as C .

Then, **for each $(t+1)$ -sized subset $I \in \mathcal{Q}$** of parties of the prior layer, P_j attempts to combine subshares (and their commitments) for the same label into a single valid share. More formally:

P_j finds the **maximum** label ℓ^* in common between all parties of I such that for the tuples $(\ell^*, \tilde{\mathbf{C}}_i^*, s_{i,j}^*, r_{i,j}^*)$, where $i \in I$, the following check ($\dagger$) holds:

- Check that $\tilde{\mathbf{C}}_i^*[i] = \text{Com}(s_{i,j}^*, r_{i,j}^*)$ and that $\tilde{\mathbf{C}}_i^*$ lies on a degree- t polynomial. Let C_i^* denote the value of this polynomial at 0.
- Check that $(0, C)$ and $\{(i, C_i^*)\}_{i \in I}$ lie on a degree- t polynomial.

If there exist no such labels, P_j continues to the next index subset. Otherwise, it combines the sub-shares into one share using Lagrange interpolation. That is:

- Let f_j^I be a degree- t polynomial s.t. $f_j^I(i) = s_{i,j}^*$ for $i \in I$. Let $s_j^{\text{new}, I} = f_j^I(0)$.
- Let g_j^I be a degree- t polynomial s.t. $g_j^I(i) = r_{i,j}^*$ for $i \in I$. Let $r_j^{\text{new}, I} = g_j^I(0)$.
- Let h_j^I be a degree- t polynomial s.t. $h_j^I(i) = \tilde{\mathbf{C}}_i^*$ for $i \in I$. Let $\mathbf{C}_j^{\text{new}, I} = h_j^I(0)$.

Finally, P_j stores $(I, \mathbf{C}_j^{\text{new}, I}, s_j^{\text{new}, I}, r_j^{\text{new}, I})$.

Analysis Idea. The idea is that if there is a label ℓ^* corresponding to a valid sharing initially stored in the old layer $\mathcal{L}_{\text{old}}$, there will again be a label I^* with a valid sharing stored in the new layer $\mathcal{L}_{\text{new}}$. Let H be a set of $t+1$ (honest) indices, and $\mathcal{L}_H \subset \mathcal{L}_{\text{old}}$ the corresponding set consisting of $t+1$ honest parties. Since $t < n/2$, such a set exists. By assumption, parties in $\mathcal{L}_H$ hold at least one valid sharing corresponding to some label ℓ . For such ℓ , and an honest sender, the requirement ($\dagger$) will be satisfied: We have that every $P_i \in \mathcal{L}_H$ holds a commitment C to the secret, a common list of commitments $\mathbf{C}^\ell = (C_1^\ell, \dots, C_n^\ell)$ consistent with C , and an opening (s_i^ℓ, r_i^ℓ) of the i -th commitment. Each $P_i \in \mathcal{L}_H$

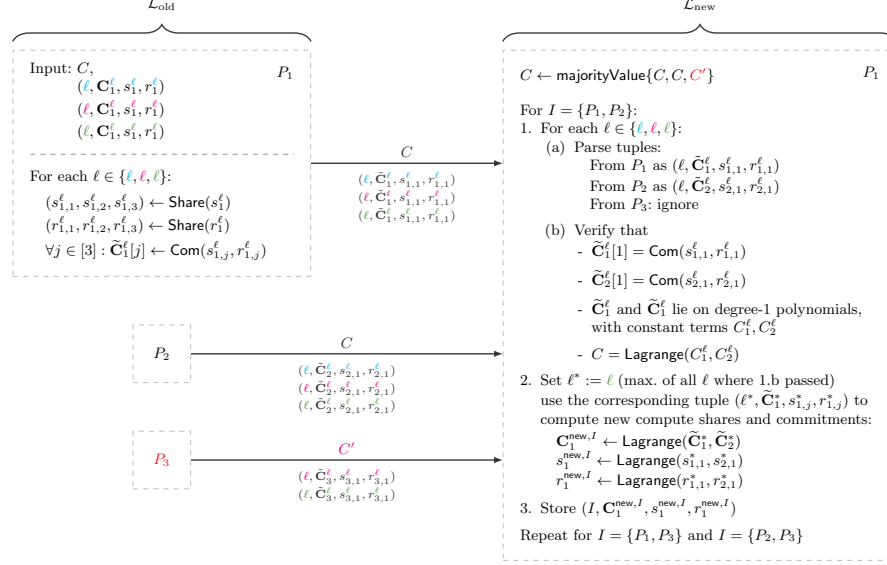


Fig. 1. Handoff protocol with $n = 3$ and $t = 1$, where $P_1 \in \mathcal{L}_{new}$ is processing set $I = \{1, 2\}$, and $P_3 \in \mathcal{L}_{old}$ is corrupted. The function **Lagrange** interpolates the value at position 0 of a degree-1 polynomial on the pair of input values at positions 1 and 2.

computes sub-shares $(s_{i,1}^\ell, \dots, s_{i,n}^\ell)$ and $(r_{i,1}^\ell, \dots, r_{i,n}^\ell)$, as well as commitments $\tilde{C}_i^\ell[j] = \text{Com}(s_{i,j}^\ell, r_{i,j}^\ell)$ and sends to each $P_j \in \mathcal{L}_{new}$ the tuple $(\ell, \tilde{C}_i^\ell, s_{i,j}^\ell, r_{i,j}^\ell)$.

Note that any honest party $P_j \in \mathcal{L}_{new}$ considers at some point the (index) subset $I = H$ (since P_j iterates over **all** index subsets of size $(t + 1)$). When doing so, all $P_j \in \mathcal{L}_{new}$, obtain the same, nonempty set of valid labels (i.e. labels where $(\dagger)$ is satisfied). Then, every honest $P_j \in \mathcal{L}_{new}$ picks the same maximum label ℓ^* , combines the received sub-shares and computes a valid share tuple while labelling it H . Since they all picked the same ℓ^* , layer $\mathcal{L}_{new}$ holds a valid sharing. Note that by pruning all tuples received from I but one, we keep the state constant when repeating handoff, thus achieving a communication that grows linearly in the number of layers.

Improving Communication Complexity. This handoff protocol enables us to construct a secure message transmission protocol without using a PKI, secure for any $t < n/2$, where the communication complexity per layer is $\Omega(\binom{n}{t+1})$. To obtain a more efficient SMT protocol with polynomial complexity in both the number of layers and the number of parties per layer, we consider the setting $(t + 1)^2 \leq n$. The idea is that, for every committee we simply choose $t + 1$ fixed, pairwise disjoint subsets of size $t + 1$ each. If only t parties are corrupt, one of these sets is guaranteed to be fully honest. Thus, it suffices to iterate I over the $t + 1$ sets, and keep $t + 1$ labels rather than $\binom{n}{t+1}$. See Section 9 for details.

3 Preliminaries

3.1 Model

Our protocols can be phrased in any existing framework for protocols with dynamic committees [36, 22, 24] that assume secure point-to-point channels. For simplicity, we describe our protocols following the Layered MPC formalization in [24], with a slightly different network model. Namely, parties do not have access to broadcast channels (we construct them from the point-to-point channels). For the ease of presentation, throughout the sections constructing the broadcast protocol, we will assume that parties can send private messages to any party in a future layer. This can be realized from channels between consecutive layers via a secure message transmission protocol (see also functionality $\mathcal{F}_{\text{smt}}$).

A layered MPC protocol can be viewed as a special case of standard MPC with a general adversary structure, specialized in the following way: 1) the interaction pattern is defined by a layered graph, and 2) the adversary can corrupt at most t parties in each layer.

Definition 1 (Layered Protocol). *Let n, t, d be positive integers. An (n, t, d) -layered protocol is a synchronous protocol Π over secure point-to-point channels with the following special features.*

- **Parties.** *There are $N = n(d + 1)$ parties partitioned into $d + 1$ layers $\mathcal{L}_i$, $0 \leq i \leq d$, where $|\mathcal{L}_i| = n$. Parties in the first layer $\mathcal{L}_0$ and the last layer $\mathcal{L}_d$ are referred to as input clients and output clients, respectively.*
- **Interaction pattern.** *The interaction consists of d rounds, where in round i parties in $\mathcal{L}_{i-1}$ may send messages to parties in $\mathcal{L}_i$ over secure point-to-point channels.*
- **Functionalities.** *We consider functionalities f that take inputs from input clients and deliver outputs to output clients.*
- **Adversaries.** *We consider adversaries who may corrupt up to t parties in each layer $\mathcal{L}_i$, $0 < i < d$. We consider active, rushing and non-retroactive adaptive adversaries¹¹.*

We say that a protocol Π is a layered protocol for $\mathcal{F}$ if it realizes $\mathcal{F}$ in the setting of general adversary structures [17, 16, 42].

3.2 Cryptographic Primitives

Signature Scheme. Some of our protocols will make use of signatures. We briefly recall the syntax of a standard signature scheme.

Definition 2. *A signature scheme Σ is a triple of efficient algorithms $\Sigma = (\text{Keygen}, \text{Sign}, \text{Vrf})$, with the following syntax:*

¹¹ For simplicity, we consider the notion of “non-retroactive” (see [22], Def. 3) adaptive adversaries, who at each round r select up to t parties from layer $\mathcal{L}_r$ to corrupt.

1. **Key Generation:** Keygen takes security parameter λ and outputs a signature-verification key pair (sk, pk) , where $\text{sk}, \text{pk} \in \{0, 1\}^\lambda$.
2. **Signature Generation:** Sign takes the message m and the signing key sk as inputs and outputs a signature σ .
3. **Verification:** Vrf takes the message m , a signature σ and the verification key pk as inputs and output a decision-bit $b \in \{0, 1\}$.

We require signatures to satisfy correctness, i.e., $\text{Vrf}(m, \text{Sign}(m, \text{sk}), \text{pk}) = 1$ for all $m \in \{0, 1\}^*$ and for $(\text{sk}, \text{pk}) = \text{Keygen}(1^\lambda)$. We also require our signatures to satisfy the standard unforgeability against chosen message attacks [38].

In our computational protocols we use any signature scheme with keys of size $O(\lambda)$ and signatures of size $O(\lambda)$ per message of $O(\lambda)$ bits [29, 55, 43, 12].

Our protocols using signatures will assume a bulletin-board PKI (also known as plain PKI). This means that every $P_i \in \mathcal{L}_k$ holds a secret key sk_i^k and all parties from all layers hold the respective public key pk_i^k . Moreover, the keys from corrupted parties can be chosen adversarially.

Commitment Scheme. We will make use of commitments in our secure message transmission protocols. We briefly recap the syntax here.

Definition 3. A non-interactive commitment scheme with message space $\mathcal{M}$, randomness space $\mathcal{R}$, and commitment space $\mathcal{C}$ consists of a triple of PPT algorithms with the following syntax: $\text{pp} \leftarrow \text{Setup}(1^\lambda)$, $C \leftarrow \text{Com}(\text{pp}, m, r)$, and $b \leftarrow \text{Open}(\text{pp}, C, m, r)$.

We require the commitment to be perfectly hiding, computationally binding and linearly homomorphic. See Definition 13 for a full definition.

Commitments with these properties can be instantiated using a variety of assumptions such as discrete-log (like Pedersen commitments [52]).

3.3 Building Blocks

Secure Message Transmission. We first formalize the functionality $\mathcal{F}_{\text{smt}}$ that allows parties to communicate securely towards future layers.

Definition 4 (Secure Message Transmission). An (n, t, d) -layered protocol Π is a t -secure message transmission protocol if it realizes $\mathcal{F}_{\text{smt}}$.

Secure Channel $\mathcal{F}_{\text{smt}}$

Public Parameters: Sender $P_{\text{sdr}} \in \mathcal{L}_0$, receiver $P_{\text{rcv}} \in \mathcal{L}_d \geq 1$. The domain M of the message. Set $m = \perp$.

- Upon receiving a message $x \in M$ from $P_{\text{sdr}} \in \mathcal{L}_0$, set $m = x$ and notify the adversary $\mathcal{A}$ that P_{sdr} sent a message. If P_{rcv} is corrupted, leak m .

- If P_{sdr} is corrupt, upon receiving m' from $\mathcal{A}$ before the end of round d , set $m = m'$ ($\mathcal{A}$ can input m' in a rushing fashion, after the honest parties in $\mathcal{L}_d$ sent their messages)
- Output m to P_{rev} at the end of round d .

Consensus and Variants. Throughout the paper we will use different variants of consensus protocols, including weak consensus, graded consensus, consensus, and their single-sender variant like graded broadcast and broadcast. As usual in this area, we defined most of these primitives using properties. However, for the case of broadcast we defined a functionality $\mathcal{F}_{\text{bc}}$ (so that it matches exactly the functionality assumed in the prior MPC papers with dynamic committees). See appendix C for a description and realization of it.

Definition 5 (Layered Weak Consensus). *Let Π be a protocol executed over $(d+1)$ layers, where each party $P_j \in \mathcal{L}_0$ holds input $x_j \in \{0, 1\}$. Parties in layer $\mathcal{L}_d$ terminate upon generating an output $y_j \in \{0, 1, \perp\}$. We say protocol Π is an (n, t, d) -layered weak consensus, if, whenever up to t out of n parties are corrupt in each layer, it satisfies the following properties:*

- **Validity:** *If there is a value x such that each honest input holder P_i in $\mathcal{L}_0$ has input $x_i = x$, then each honest party P_j in $\mathcal{L}_d$ outputs $y_j = x$.*
- **Consistency:** *There exists a value y , such that every honest party in $\mathcal{L}_d$ outputs either y or $\perp$.*

Definition 6 (Layered Graded Consensus). *Let Π be a protocol executed over $(d+1)$ layers, where each party $P_j \in \mathcal{L}_0$ holds input $x_j \in \{0, 1\}$. Parties in layer $\mathcal{L}_d$ terminate upon generating an output $(y_j, g_j) \in (\{0, 1\} \times \{1, 2\}) \cup \{(\perp, 0)\}$. We say protocol Π is an (n, t, d) -layered graded consensus, if, whenever up to t out of n parties are corrupt in each layer, it satisfies the following:*

- **Validity:** *If there is a value x such that each honest input holder in $\mathcal{L}_0$ has input $x_j = x$, then each honest party in $\mathcal{L}_d$ outputs $(x, 2)$.*
- **Graded Consistency:** *If some honest $P_i \in \mathcal{L}_d$ outputs $(y_i, 2)$, then for every honest $P_j \in \mathcal{L}_d$ we have $y_j = y_i$ and $g_j \geq 1$. Moreover, if some honest $P_i \in \mathcal{L}_d$ outputs $(y_i, 1)$, no honest party $P_j \in \mathcal{L}_d$ outputs $(y_j, 1)$ with $y_j \neq y_i$.*

Definition 7 (Layered Graded Broadcast). *Let Π be a protocol executed over $(d+1)$ layers, where a designated party $P_s \in \mathcal{L}_0$ holds input $x \in \{0, 1\}$. Parties in layer $\mathcal{L}_d$ terminate upon generating an output $(y_j, g_j) \in (\{0, 1\} \times \{1, 2\}) \cup \{(\perp, 0)\}$. We say protocol Π is an (n, t, d) -layered graded broadcast if, whenever up to t out of n parties are corrupt in each layer, it satisfies the following:*

- **Validity:** *If $P_s \in \mathcal{L}_0$ is honest, then every honest P_i outputs $(x, 2)$.*
- **Graded Consistency:** *If some honest $P_i \in \mathcal{L}_d$ outputs $(y_i, 2)$, then for every honest $P_j \in \mathcal{L}_d$ we have $y_j = y_i$ and $g_j \geq 1$. Moreover, if some honest $P_i \in \mathcal{L}_d$ outputs $(y_i, 1)$, no honest party $P_j \in \mathcal{L}_d$ outputs $(y_j, 1)$ with $y_j \neq y_i$.*

Definition 8 (Layered Consensus). Let Π be a protocol executed over $d + 1$ layers, where each party $P_j \in \mathcal{L}_0$ holds input $x_j \in \{0, 1\}$. Parties in layer $\mathcal{L}_d$ terminate upon generating an output $y_j \in \{0, 1\}$. We say protocol Π is an (n, t, d) -layered consensus protocol if whenever up to t out of n parties are corrupt in each layer, it satisfies the following properties:

- **Validity:** If there is a value x such that each honest input holder $P_i \in \mathcal{L}_0$ has input $x_i = x$, then each honest party $P_j \in \mathcal{L}_d$ outputs $y_j = x$.
- **Consistency:** There exists a value y , such that every honest party in $\mathcal{L}_d$ outputs y .

Verifiable Secret Sharing. We also define layered versions for verifiable secret sharing and the moderated version.

Definition 9 (Verifiable Secret Sharing). An (n, t, d, d') -layered verifiable secret sharing (VSS) protocol consists of two subprotocols:

- A protocol **Share** runs over d layers $\mathcal{L}_0, \dots, \mathcal{L}_d$, where a designated dealer $P_s \in \mathcal{L}_0$ holds an input s , and every P_i in $\mathcal{L}_d$ outputs a value sh_i .
- A protocol **Reconstruct** executed over d' layers $\mathcal{L}_d, \dots, \mathcal{L}_{d+d'}$. At the end of the protocol, every honest party $P_i \in \mathcal{L}_{d+d'}$ outputs a value s' .

Such that the following is satisfied:

- There is a unique value s' defined by the joint views of all honest parties in $\mathcal{L}_d$ such that (1) if P_s is honest, $s' = s$ and the joint view of all corrupt parties in layers $\mathcal{L}_0, \dots, \mathcal{L}_d$ is independent of s and (2) after executing the protocol **Reconstruct**, every honest party in $\mathcal{L}_{d+d'}$ outputs s' .

We generalize *moderated VSS* (*mVSS*) [44] to the layered setting:

Definition 10 (Layered Moderated VSS). An (n, t, d, d') -layered moderated VSS (*lmVSS*) protocol consists of two subprotocols:

- A protocol **Share** executed over d layers $\mathcal{L}_0, \dots, \mathcal{L}_d$, with a designated party $P_s \in \mathcal{L}_0$, called the dealer holding some input s , as well as a sequence of designated parties $P_{m_1}, \dots, P_{m_k}$, called the moderators, with $P_{m_i} \in \mathcal{L}_{l_i}$ for some $l_1 < \dots < l_k < d$, where every P_i in $\mathcal{L}_d$ outputs a value sh_i and a flag $\text{success}_i \in \{0, 1\}$.
- A protocol **Reconstruct** executed over d' layers $\mathcal{L}_d, \dots, \mathcal{L}_{d+d'}$. At the end of the protocol, every honest party $P_i \in \mathcal{L}_{d+d'}$ outputs a value s' .

Such that the following is satisfied:

- If all moderators are honest, every honest P_i outputs $\text{success}_i = 1$ after the protocol **Share**.
- If some honest P_i outputs $\text{success}_i = 1$ after running **Share**, then there is a unique value s' defined by the joint views of all honest parties in $\mathcal{L}_d$ such that (1) if P_s is honest, $s' = s$ and the joint view of all corrupt parties in layers $\mathcal{L}_0, \dots, \mathcal{L}_d$ is independent of s and (2) after executing the protocol **Reconstruct**, every honest party in $\mathcal{L}_{d+d'}$ outputs s' .

4 Layered Weak and Graded Consensus

Proofs for this section can be found in appendix A.

4.1 Perfect Security for $t < n/3$

For completeness, we describe protocols from [7, 32] that achieve layered weak consensus and layered graded consensus and prove them secure, see Figures 2 and 3. We prove their security in appendix A.

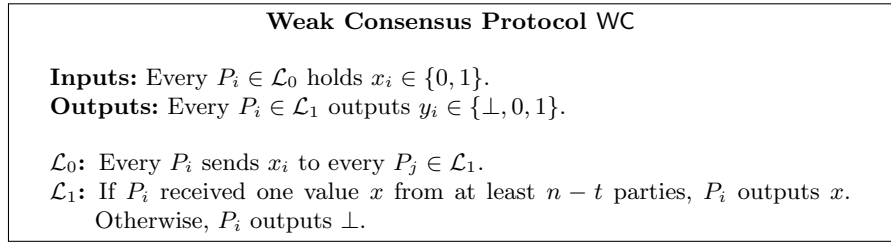


Fig. 2. Weak Consensus for $t < n/3$

Theorem 5. *Protocol WC defined in Figure 2 is an $(n, t, 1)$ -layered weak consensus protocol for any $t < \frac{n}{3}$.*

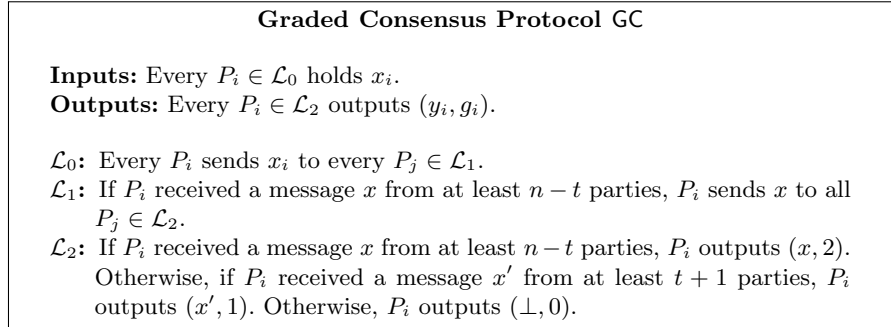


Fig. 3. Graded Consensus for $t < n/3$

Theorem 6. *Protocol GC defined in Figure 3 is an $(n, t, 2)$ -layered graded consensus protocol for any $t < \frac{n}{3}$.*

4.2 Computational Security for $t < n/2$

We now modify weak and graded consensus schemes [49, 33, 44] to get security in the layered setting assuming a plain PKI (see Appendix A for security proofs).

In the following, we let a **degree-0-certificate** for $b \in \{0, 1\}$ be a list of valid signatures on b by at least $n - t$ parties in $\mathcal{L}_0$. We define a **degree- k -certificate** for $b \in \{0, 1\}$ to be a list $(c_i, \sigma_i)_{i \in \mathcal{I}}$ such that each c_i is a degree- $(k-1)$ -certificate for b , each σ_i is a valid signature on c_i by $P_i \in \mathcal{L}_k$ and $|\mathcal{I}| \geq n - t$.

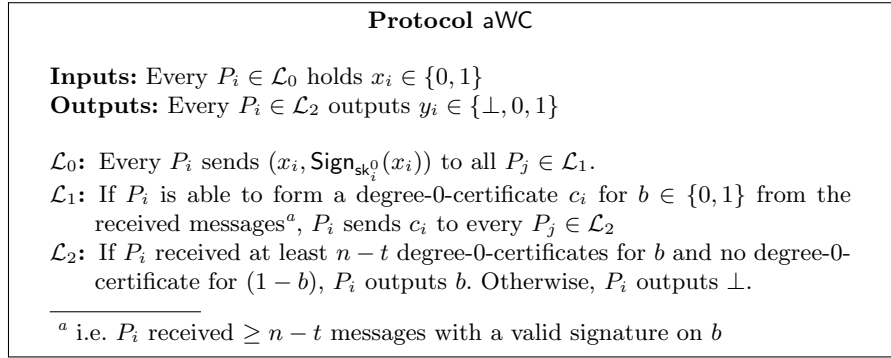


Fig. 4. Authenticated Weak Consensus for $t < n/2$

Theorem 7. *Assuming a plain PKI, protocol aWC defined in Figure 4 is an $(n, t, 2)$ -layered weak consensus protocol for any $t < \frac{n}{2}$.*

We now introduce an authenticated layered graded consensus protocol based on the graded broadcast protocol in [49], but achieving a slightly stronger consistency level at the expense of an extra round.

Theorem 8. *Assuming a plain PKI, protocol aGC defined in Figure 5 is an $(n, t, 4)$ -layered graded consensus protocol for any $t < \frac{n}{2}$.*

4.3 Graded Broadcast Towards Multiple Layers

First, note that graded broadcast can be achieved from graded consensus simply by letting the sender $P_{\text{sdr}} \in \mathcal{L}_0$ send its input to all parties in the next layer $\mathcal{L}_1$, who will then jointly execute a graded consensus protocol. Therefore, using theorem 6 and 8, we obtain the following corollary:

Corollary 5. *There is an $(n, t, 3)$ -layered graded broadcast protocol for any $t < n/3$. Moreover, assuming a plain PKI for signatures, there is an $(n, t, 5)$ -layered graded broadcast protocol for any $t < n/2$.*

Authenticated Graded Consensus Protocol aGC

Inputs: Every $P_i \in \mathcal{L}_0$ holds x_i .

Outputs: Every $P_i \in \mathcal{L}_3$ outputs (y_i, g_i) .

$\mathcal{L}_0$: Every P_i sends $(x_i, \text{Sign}_{\text{sk}_i^0}(x_i))$ to all $P_j \in \mathcal{L}_1$.

$\mathcal{L}_1$: If P_i is able to form a degree-0-certificate c_i for $b \in \{0, 1\}$ from the received messages, P_i sends $(c_i, \text{Sign}_{\text{sk}_i^1}(c_i))$ to every $P_j \in \mathcal{L}_2$

$\mathcal{L}_2$: If P_i received at least $n - t$ degree-0-certificates for b (with valid signatures) and no degree-0-certificate for $(1 - b)$, P_i forms a degree-1-certificate c_i and sends $(c_i, \text{Sign}_{\text{sk}_i^2}(c_i))$ to every $P_j \in \mathcal{L}_3$

$\mathcal{L}_3$: If P_i received at least $n - t$ degree-1-certificates for b (with valid signatures) and no degree-1-certificate for $(1 - b)$, P_i forms a degree-2-certificate c_i and sends $(c_i, \text{Sign}_{\text{sk}_i^3}(c_i))$ to every $P_j \in \mathcal{L}_4$

$\mathcal{L}_4$: If P_i received at least $n - t$ degree-2-certificates for b and none for $(1 - b)$, P_i outputs $(b, 2)$.

Otherwise, if P_i received at least one degree-2-certificates for b and none for $(1 - b)$, P_i outputs $(b, 1)$.

Otherwise, P_i outputs $(\perp, 0)$.

Fig. 5. Authenticated Graded Consensus Protocol for $t < n/2$

In Section 5, we assume graded broadcast (a.k.a. gradecast) channels which deliver output towards two committees with graded consistency holding across these committees. We briefly discuss a technique to achieve this. A similar method is used to propagate the output of a weak consensus protocol from one layer to subsequent ones, see Section 7.

Proposition 1. *Assume $t < n/2$. Let π be an (n, t, d) -layered gradecast protocol and $d' > d$. Then there exists an (n, t, d') -layered protocol π' which achieves gradecast towards $\mathcal{L}_d \cup \mathcal{L}_{d'}$, with graded consistency holding for any pair of honest parties $P, P' \in \mathcal{L}_d \cup \mathcal{L}_{d'}$.*

5 Moderated Layered VSS

We now present a transformation that converts any layered VSS protocol Π relying on (layered) broadcast into a moderated layered VSS protocol Π' that uses only a layered graded broadcast channels GradeCast_ℓ and $\text{GradeCast}_{\ell, 2\ell}$, generalizing the construction from [44]. Here, GradeCast_ℓ delivers outputs at layer ℓ , and $\text{GradeCast}_{\ell, 2\ell}$ delivers outputs at layers ℓ and 2ℓ (assuming that the gradecasts start at layer 0) (see §4.3).

Protocol Overview. Informally, the transformation of the sharing phase Share of protocol Π works as follows. For every committee $\mathcal{L}_k$:

- If no broadcast channel is used in Π from $\mathcal{L}_k$ to $\mathcal{L}_{k+1}$, then parties in $\mathcal{L}_k$ proceed as in Share .

- If some parties in $\mathcal{L}_k$ of Π broadcast a message, we “expand” the protocol by 2ℓ layers, and use a moderator in an intermediate committee $\mathcal{L}_\ell$ to simulate these broadcasts (see Figure 6). This process is not guaranteed to work, but if it fails, a warning bit $y = 1$ will be sent to parties in the final committee of Π ’s sharing phase. Furthermore, the private messages that parties in Π would normally send from $\mathcal{L}_k$ to $\mathcal{L}_{k+1}$ are instead sent to the corresponding recipients 2ℓ layers down the line.
- Parties in the final committee output **success** = 0 iff they received a warning (i.e. $y = 1$) from $t + 1$ parties (or more) from within any one committee.

Intuitively, this transformation works because the moderator can neither alter an honest sender’s message nor try to distribute an inconsistent message without all honest parties detecting this based on the gradecast outputs.

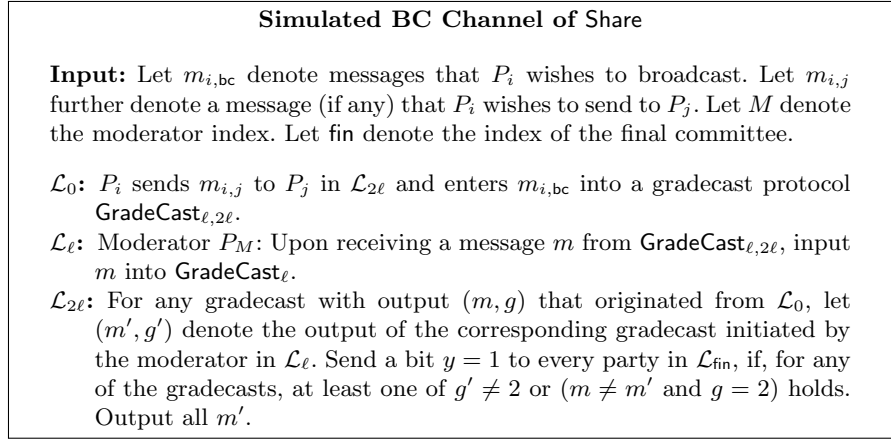


Fig. 6. Subprotocol for BC Channels

More formally, let

$$(\{m_{i,j}\}_{j \in [n]}, m_{\text{bc}}) = \text{NextMessage}_\Pi(i, k, r_i, S_i)$$

denote the next message function of Π which specifies the behaviour of party P_i in layer $\mathcal{L}_k$, $\text{fin} > k \geq 0$. Above, i denotes the index of the party, k denotes the layer, r_i the randomness, S_i the set of received messages, $m_{i,j}$ the message that P_i sends to P_j from the next layer, m_{bc} the message that P_i broadcasts.

We further let $s_i = \text{OutputShare}_\Pi(i, r_i, S_i)$ denote the function which specifies the behaviour of party P_i in the final sharing layer of Π . Above, s_i denotes the VSS share, and **success** $_i$ the VSS success flag.

We describe the transformation in Figure 7. For ease of presentation, we assume that every layer of Π is expanded into 2ℓ layers (in practice, this is only necessary for layers with broadcasts). Our compiler achieves the following result:

Moderated Layered VSS - Sharing Phase

Public Parameters: Let $|\text{Share}|$ denote the number of layers in the sharing phase of Π . Let $\text{fin} = |\text{Share}| - 1$ denote the final layer of Π 's sharing phase. Let $M = (M^0, \dots, M^{|\text{Share}|-1})$ denote the moderator index sequence.

$\mathcal{L}_0$: Let r_i denote the randomness of P_i . Each P_i executes

$$(\{m_{i,j}\}_{j \in [n]}, m_{i,\text{bc}}) = \text{NextMessage}_\Pi(i, 0, r_i, \perp),$$

P_i enters $m_{i,\text{bc}}$ into a gradecast protocol $\text{GradeCast}_{\ell, 2\ell}$ and sends $m_{i,j}$ to P_j in $\mathcal{L}_{2\ell}$.

$\mathcal{L}_{2\ell k + \ell, \text{fin} > k \geq 0}$: Party P_{M^k} : Upon receiving a message m from $\text{GradeCast}_{\ell, 2\ell}$, input m into GradeCast_ℓ .

$\mathcal{L}_{2\ell k, \text{fin} > k > 0}$: For any gradecast $\text{GradeCast}_{\ell, 2\ell}$ that started out from layer $\mathcal{L}_{2\ell(k-1)}$, let (m_i, g_i) denote the output of party P_i in $\mathcal{L}_{2\ell k}$, and let (m'_i, g'_i) denote P_i 's output of the corresponding gradecast GradeCast_ℓ that was initiated by the moderator in $\mathcal{L}_{2\ell k - \ell}$. Let $m'_{i,\text{bc}}$ denote the set of all messages m'_i received by P_i in this way. Let $m_{j,i}$ further denote a message that P_i received from party P_j of layer $\mathcal{L}_{2\ell(k-1)}$ (if any). (†) Each P_i does the following:

- P_i sends a warning bit $y = 1$ to each party of the final committee $\mathcal{L}_{2\ell \text{fin}}$, if for any of the gradecasts, at least one of $g'_i \neq 2$ or $(m_i \neq m'_i \text{ and } g_i = 2)$ holds.
- Additionally, P_i executes

$$(\{m^*_{i,j}\}_{j \in [n]}, m^*_{i,\text{bc}}) = \text{NextMessage}_\Pi(i, k, r_i, \{m_{j,i}\}_{j \in [n]} \cup m'_{i,\text{bc}}),$$

where r_i denote the randomness of P_i . P_i then enters $m^*_{i,\text{bc}}$ into a gradecast protocol $\text{GradeCast}_{\ell, 2\ell}$ and sends $m^*_{i,j}$ to P_j in $\mathcal{L}_{2\ell(k+1)\ell}$.

$\mathcal{L}_{2\ell \text{fin}}$: Let $m_{j,i}$, $m'_{i,\text{bc}}$ and y denote the messages received resp. the warning bit computed by party P_i in the same way as in (†). Let P_i execute

$$s_i = \text{OutputShare}_\Pi(i, r_i, \{m_{j,i}\}_{j \in [n]} \cup m'_{i,\text{bc}}),$$

where r_i is randomness of P_i . Further, let $y_{j,i,a}$ denote the warning bit sent to party P_i by party P_j in layer $\mathcal{L}_a$.

Then, P_i outputs $(s_i, \text{success} = 1)$, if $y = 0$ and for every layer $\mathcal{L}_a$, P_i received at most t warning bits $y_{j,i,a} = 1$. Otherwise, P_i outputs $(\perp, \text{success} = 0)$.

Fig. 7. Compiler: From Layered VSS with Broadcast to Moderated Layered VSS with Gradecast (No Broadcast).

Theorem 9. Assume there exists a layered VSS protocol Π , tolerating up to $t < n/2$ corruptions, which may additionally use a broadcast channel in its sharing phase. Then there exists a moderated layered VSS protocol Π' , which also tolerates up to t corruptions and only uses a gradecast channel.

Proof. Let Π be a layered VSS protocol tolerating up to $t < n/2$ corruptions. We now show that applying our compiler (Figure 7) to Π yields a protocol Π' that is a moderated VSS protocol tolerating up to t corruptions.

First, we show that (i) if all moderators P_{M^k} , $\text{fin} > k \geq 0$, are honest, then every honest P_i in $\mathcal{L}_{2\ell\text{fin}}$ outputs $\text{success}_i = 1$. By protocol description, $\text{success}_i = 0$ only if P_i computes $y = 1$ or P_i received at least $t + 1$ warning bits $y_{j,i,a} = 1$ from parties in some committee $\mathcal{L}_a$. Thus, to prove claim (i), it suffices to show that if all moderators are honest, no honest party computes a warning bit $y = 1$: An honest party in layer $\mathcal{L}_{2\ell k}$ ($\text{fin} \geq k > 0$) computes $y = 1$ if and only if for some gradedcast GradeCast_ℓ it holds that $g'_i \neq 2$, or alternatively ($m_i \neq m'_i$ and $g_i = 2$). Whenever P_{M^k} is honest, by validity of GradeCast_ℓ we get that every honest party P_i in $\mathcal{L}_{2\ell k}$ outputs $g'_i = 2$. Further, if $g_i = 2$ for some honest party P_i in $\mathcal{L}_{2\ell k}$, by the graded consistency of $\text{GradeCast}_{\ell,2\ell}$, we get that every honest party P_j in layers $\mathcal{L}_{2\ell k}$ and $\mathcal{L}_{2\ell k-\ell}$ receives the same output $m = m_i$ from $\text{GradeCast}_{\ell,2\ell}$. In particular, the moderator P_{M^k} in $\mathcal{L}_{2\ell k-\ell}$ also receives m . If P_{M^k} is honest, by validity of GradeCast_ℓ we then get that every honest party P_i in $\mathcal{L}_{2\ell k}$ will output $m'_i = m$. Thus, if P_{M^k} is honest, no honest party from $\mathcal{L}_{2\ell k}$ computes $y = 1$. This proves (i).

Next, we show that (ii) if an honest P_i outputs $\text{success}_i = 1$, then our protocol preserves the layered VSS properties. Note that if an honest party P_i outputs $\text{success}_i = 1$ at the end of the sharing phase, then from every layer $\mathcal{L}_{2\ell k}$, $\text{fin} > k > 0$, P_i received at most t warning bits $y_{j,i,2\ell k} = 1$. As $t < n/2$, we get that in every layer $\mathcal{L}_{2\ell k}$, there is at least one honest party P_i that did not send a warning bit to the final committee. Hence, (by the protocol description) it must be that $g'_i = 2$ for this P_i . By graded consistency, we get that every honest party P_j in the same layer obtained $m'_j = m'_i$ (thus, our compiler preserves the consistency of the broadcast used in Π). Further, as for the gradedcasts initiated by honest parties every P_j outputs $g_j = 2$ and $m_j = m$ (where m was the message gradedcasted by the honest sender), it must be that $m_i = m'_i$ (otherwise the honest P_i would have sent a warning bit). Together, we get $m'_j = m'_i = m_i = m$. Thus, our compiler preserves the validity of the broadcasts used in Π . Hence, in this case we achieve the same properties as the original VSS. $\square$

Instantiating this construction with the VSS protocol from [24], Sec. 5, resp. with the one from [25], Sec. 5 and from [26], Sec. 8, as well as the gradedcast protocols discussed in Sec. 4, we obtain the following results:

Corollary 6. *There exists a perfectly secure $(n, t, 14, 1)$ -layered moderated VSS protocol, which uses 3 moderators for $t < n/3$.*

Corollary 7. *Assuming plain PKI for signatures and non-interactive homomorphic commitments, there exists a cryptographically secure $(n, t, 16, 1)$ -layered moderated VSS with 2 moderators for $t < n/2$.*

Corollary 8. *Assuming plain PKI for signatures, there exists a cryptographically secure $(n, t, 60, 2)$ -layered moderated VSS with 15 moderators for $t < n/2$.*

6 Random Coin from Moderated VSS

We now adapt the oblivious leader election protocol from [44] to construct a so-called *weak layered random coin protocol* assuming the existence of a constant-round layered weak consensus protocol and a constant round layered moderated VSS protocol. Formally, we define a weak layered coin protocol as follows:

Definition 11 (Weak Layered Random Coin). *A protocol Π over $d_1 + d_2$ layers is a (n, t, d_1, d_2) statistically (resp., computationally) secure weak layered random coin protocol with domain $\mathcal{D}$ and success probability δ if, for any unconditional (resp., PPT) adversary $\mathcal{A}$ corrupting up to t parties in each layer, the probability that all honest parties output the same value is at least δ , and, for any $b, b' \in \mathcal{D}$, the view of $\mathcal{A}$ during the first d_1 rounds given that all honest parties output b is (computationally) indistinguishable from the case where all honest parties output b' .*

Theorem 10. *Assuming an (n, t, d_1, d_2) -layered moderated VSS using k moderators and an (n, t, d_2) -layered weak consensus, protocol Coin (Fig. 8) is a weak (t, d_1, d_2) -layered coin with domain $\mathbb{Z}_q$ and success probability $\delta := 0.99 \cdot \left(\frac{n-t}{n}\right)^k$.*

Proof. After round d_1 , every moderator sequence $\mathbf{j}$ is either disqualified, i.e. for some $i \in [n]$, every honest party in $\mathcal{L}_{d_1}$ has obtained `success` = 0 from `Sharei,j`, or the moderator sequence acted correctly, in which case the properties of moderated VSS guarantee that there exists a value $(a_j, b_j) \in \mathbb{Z}_{100 \cdot n^{2k}} \times \mathbb{Z}_q$ such that every honest $P_i \in \mathcal{L}_{d_1+d_2}$ will compute $(a_j^{(i)}, b_j^{(i)}) = (a_j, b_j)$, no matter what the corrupt parties in $\mathcal{L}_{d_1}, \dots, \mathcal{L}_{d_1+d_2}$ do. Moreover, (a_j, b_j) is independent of the view of the adversary up until round d_1 and, as some honest parties input uniformly random summands, uniformly distributed. Thus, if we denote by $\mathcal{T} \subseteq [n]^k$ the set of moderator sequences which are not disqualified in the above sense, there is a probability of at least $\delta = 0.99 \cdot \left(\frac{n-t}{n}\right)^k$ that all the $(a_j)_{j \in \mathcal{T}}$ are pairwise distinct and that the minimum of them corresponds to a fully honest moderator sequence $\mathbf{j}^*$. In this event, all honest parties in $\mathcal{L}_{d_1+d_2}$ are guaranteed to output $b_{\mathbf{j}^*}$, which is uniformly random given the view of the adversary until round d_1 . $\square$

Instantiating this construction with the protocols from Sections 4 and 5, we obtain the results below. Note that in the $t < n/2$ case, our simplifying assumption that the reconstruction phase and weak consensus have equal round complexity d_2 is not met. This can be easily addressed, e.g. by having $\mathcal{L}_{d_1}$ and $\mathcal{L}_{d_1+2}$ execute `Reconstruct` (instead of $\mathcal{L}_{d_1}$ and $\mathcal{L}_{d_1+1}$).

Corollary 9. *There exists a weak $(n, t, 14, 1)$ -layered coin protocol with perfect unpredictability and success probability $\delta \geq 0.29$, for $t < n/3$.*

Corollary 10. *Assuming a PKI for signatures and non-interactive homomorphic commitments, there exists a weak $(n, t, 16, 2)$ -layered coin protocol with computational unpredictability and success probability $\delta \geq 0.24$, for $t < n/2$.*

Protocol Coin

Public Parameters: (n, t, d_1, d_2) -layered moderated VSS protocol (Share, Reconstruct); $k < d_1$: the number of moderators used in Share; (n, t, d_2) -layered weak consensus protocol WC; Number of coin sides q .

$\mathcal{L}_0$: Every P_i chooses independent uniformly random values $a_{i,j} \in \mathbb{Z}_{100 \cdot n^{2k}}$ and $b_{i,j} \in \mathbb{Z}_q$ for each $j \in [n]^k$. Then, for each $i \in [n]$ and $j \in [n]^k$, all parties execute a layered moderated VSS protocol instance, denoted $\text{Share}_{i,j}$, where j is the sequence of moderators and P_i is the dealer with input $(a_{i,j}, b_{i,j})$

$\mathcal{L}_1, \dots, \mathcal{L}_{d_1-1}$: All parties participate in $\text{Share}_{i,j}$ for all $i \in [n]$ and $j \in [n]^k$

$\mathcal{L}_{d_1}$: For every $j \in [n]^k$, one instance of weak consensus (denoted WC_j) is started. P_i inputs 1 into WC_j if P_i output $\text{success} = 1$ from $\text{Share}_{i,j}$ for every $l \in [n]$, otherwise P_i inputs 0.

Additionally, for every $i \in [n]$, $j \in [n]^k$, all parties start a protocol instance $\text{Reconstruct}_{i,j}$ to reconstruct the secret shared in $\text{Share}_{i,j}$.

$\mathcal{L}_{d_1+1}, \dots, \mathcal{L}_{d_1+d_2-1}$: All parties participate in $\text{Reconstruct}_{i,j}$ and WC_j for all $i \in [n]$ and $j \in [n]^k$.

$\mathcal{L}_{d_1+d_2}$: Write $(a_{i,j}^{(l)}, b_{i,j}^{(l)})$ for P_l 's outputs from $\text{Reconstruct}_{i,j}$ and $\text{success}_j^{(l)}$ for the value P_l outputs from WC_j . Further define

$$a_j^{(l)} := \sum_{i \in [n]} a_{i,j}^{(l)} \mod 100 \cdot n^{2k},$$

$$b_j^{(l)} := \sum_{i \in [n]} b_{i,j}^{(l)} \mod q$$

Finally, P_l outputs $b_{j^{(l)}}^{(l)}$, where $j^{(l)}$ is defined as

$$j^{(l)} := \text{argmin}\{a_j^{(l)} \mid j \in [n]^k, \text{success}_j^{(l)} = 1\}$$

(If there is no unique minimum, P_j outputs a random value).

Fig. 8. Random coin protocol from weak consensus and moderated VSS

Corollary 11. *Assuming a PKI for signatures, there exists a weak $(n, t, 60, 2)$ -layered coin protocol with computational unpredictability and success probability $\delta \geq 0.99 \cdot (1/2)^{15}$, for $t < n/2$.*

7 Layered Consensus

We now introduce our layered consensus protocol. In the following we assume a weak (n, t, d_1, d_2) -layered coin protocol Coin with success probability $\delta > 0$, and a weak consensus protocol WC running over d_1 layers. We write $D := d_1 + d_2$.

Theorem 11. *The protocol LayeredConsensus achieves consensus with probability $1 - (1 - \delta/2)^\lambda$.*

Protocol LayeredConsensus

Inputs: Every $P_i \in \mathcal{L}_0$ holds $x_i \in \{0, 1\}$

Public Parameters: Security parameter λ , (n, t, d_1) -layered weak consensus protocol WC , (n, t, d_1, d_2) -layered weak coin protocol Coin

$\mathcal{L}_0$: Input x_i into WC_1 , start Coin_1

for $\ell = 0, \dots, \lambda$:

$\mathcal{L}_{\ell D+1}, \dots, \mathcal{L}_{\ell D+(d_1-1)}$: Participate in WC_ℓ and Coin_ℓ

$\mathcal{L}_{\ell D+d_1}$: Participate in Coin_ℓ . Additionally, every P_i sends their output from WC_ℓ to all parties in $\mathcal{L}_{\ell D+(d_1+d_2)}$,

$\mathcal{L}_{\ell D+(d_1+1)}, \dots, \mathcal{L}_{\ell D+(d_1+d_2-1)}$: Participate in Coin_ℓ

$\mathcal{L}_{\ell D+(d_1+d_2)}$: Let y_i^ℓ denote the median of all values P_i received from parties in $\mathcal{L}_{\ell D+d_1}$ (using $0 < \perp < 1$). Define x_i^ℓ to be the output from Coin_ℓ if $y_i^\ell = \perp$, and $x_i^\ell := y_i^\ell$ otherwise.

Then, if $\ell < \lambda$, P_i inputs x_i^ℓ into $\text{WC}_{\ell+1}$ and starts $\text{Coin}_{\ell+1}$.

Otherwise, P_i terminates with output x_i^λ .

Fig. 9. Layered consensus protocol

Proof. We first show validity: If all honest $P_i \in \mathcal{L}_0$ input $x_i = b$ for some $b \in \{0, 1\}$, then by validity of WC , every honest party in $\mathcal{L}_{d_1}$ will output b from WC_1 . Since $t < n/2$, this implies that every honest P_i in $\mathcal{L}_{(d_1+d_2)}$ will compute $x_i^1 = b$. By induction, the same holds for all x_i^ℓ computed by honest P_i in $\mathcal{L}_{(\ell+1)D}$ for $\ell = 1, \dots, \lambda$.

Consistency can be shown as follows: For any $\ell = 0, \dots, \lambda$, the values y_i^ℓ computed by honest parties in $\mathcal{L}_{\ell D+(d_1+d_2)}$ are guaranteed to be in the set of output values honest parties got from WC_ℓ (since $t < n/2$). Therefore, they are either all $\perp$ (if every honest party output $\perp$ from WC_ℓ) or they are contained in a set $\{b, \perp\}$ for some $b \in \{0, 1\}$ which was determined by what happened up to $\mathcal{L}_{\ell D+d_1}$. Therefore, the probability that all honest parties in $\mathcal{L}_{\ell D+(d_1+d_2)}$ output b from the Coin_ℓ is at least $\delta/2$. If this happens, the argument used in proving validity of LayeredConsensus proves that all honest parties in $\mathcal{L}_{(\ell+1)D}$ output b . Finally, the probability that this happens at least once is $1 - (1 - \frac{\delta}{2})^\lambda$. $\square$

By instantiating this construction with our protocols from Sections 4 and 6, we obtain the following results:

Corollary 12. *There exist layered consensus protocols with (1) statistical security for $t < n/3$ and no setup assumptions and (2) cryptographic security for $t < n/2$ assuming a plain PKI.*

Note that as in the previous section, our assumption that the weak consensus protocol runs for d_1 rounds is not met here. This can be addressed by a similar technique as discussed in Sec. 4.3.

8 Impossibility for $t \geq n/2$

We show the impossibility of layered broadcast for $t \geq n/2$. This is in contrast to the classical setting with a fixed number of parties, where broadcast is possible for any number of corruptions $t < n$ [28].

Theorem 12. *For $t \geq n/2$ corruptions, layered broadcast with a fixed number of rounds and error probability below $\frac{1}{4}$ is impossible.*

Proof. Assume there exists a (possibly authenticated, possibly randomized) broadcast protocol Π running over k layers and tolerating $t \geq n/2$ corruptions, such that validity and consistency hold with probability $1 - \epsilon$ for some $\epsilon < \frac{1}{4}$.

Consider the following four execution scenarios:

- A The sender $P_{\text{sdr}} \in \mathcal{L}_0$ is honest with input 0. In every layer, there are t corrupt parties who do nothing.
- B The sender $P_{\text{sdr}} \in \mathcal{L}_0$ is corrupt, and pretends to be honest with input 0 towards all honest parties. In layers $\mathcal{L}_1, \dots, \mathcal{L}_{k-1}$, there are t corrupt parties each, $n - t$ of which participate (together with P_s) in a simulated parallel execution Π with input 1 by P_s . Communication with the honest parties is blocked, up until the second-to-last layer, where all corrupt parties send the messages from their simulated execution to one honest party (say $P_1 \in \mathcal{L}_k$).
- C Analogous to B, but with the input values switched, i.e. P_{sdr} sends 1 to the honest parties, and 0 in the simulated execution of Π .
- D Analogous to A but with input 1.

By validity, all honest parties must output 0 in scenario A with probability $\geq 1 - \epsilon$. Since all honest parties in $\mathcal{L}_k$ (except P_1) have identical views in executions A and B, they must output 0 in scenario B with probability $\geq 1 - \epsilon$ as well. Thus, by consistency, P_1 outputs 0 in scenario B with probability $\geq 1 - 2\epsilon$. But P_1 has identical views in scenario B and C, so by the same argument all other honest parties in $\mathcal{L}_k$ must output 0 in scenario C with probability $\geq 1 - 3\epsilon$. Finally, all honest parties in $\mathcal{L}_k$ except P_1 have identical views in scenarios C and D, so honest parties should output 0 with probability $\geq 1 - 3\epsilon$ in D as well. For $\epsilon < \frac{1}{4}$, this contradicts validity as honest parties must output 1 with probability $1 - \epsilon > 3\epsilon$ in scenario D. $\square$

9 Protocol SMT for Secure Message Transmission

We now describe our secure message transmission protocol SMT, defined in Figure 10, which realizes the functionality $\mathcal{F}_{\text{smt}}$ (Definition 4). At a high level, the scheme lets the sender $P_{\text{sdr}} \in \mathcal{L}_0$ secret-share the original secret message using a Shamir-sharing protocol with commitments to shares. Every intermediate layer performs a resharing of the secret by distributing sub-shares with commitments, and the receiving layer performs a non-trivial method to select subsets of sub-shares to combine into new shares without increasing the state size at each step. Finally, the second to last layer reconstructs the message towards $P_{\text{rcv}} \in \mathcal{L}_d$.

We prove the following result in Appendix B.2.

Protocol SMT	
Inputs:	A sender $P_{\text{sdr}} \in \mathcal{L}_0$ holds a secret $s \in \mathbb{F}_p$, Receiver $P_{\text{rcv}} \in \mathcal{L}_d$.
$\mathcal{L}_0, \mathcal{L}_1$:	P_{sdr} shares s towards $\mathcal{L}_1$ using the protocol Share (Figure 11).
$\mathcal{L}_r, \mathcal{L}_{r+1}$ ($1 \leq r \leq d-2$):	Parties from the “old” layer $\mathcal{L}_r$ and the “new” layer $\mathcal{L}_{r+1}$ execute the protocol Handoff (Figure 13).
$\mathcal{L}_{d-1}, \mathcal{L}_d$:	Parties of layer $\mathcal{L}_{d-1}$ reconstruct s towards $P_{\text{rcv}} \in \mathcal{L}_d$ using the Open protocol (Figure 12)

Fig. 10. Secure Message Transmission protocol

Theorem 13. *Assuming non-interactive homomorphic commitments, protocol SMT realizes $\mathcal{F}_{\text{smt}}$ for d layers with communication $O(d \cdot \binom{n}{t+1} \cdot n^3)$ and is secure up to $t < n/2$ corruptions per layer*

Optimization. Protocol SMT as presented has a communication complexity $\Omega(\binom{n}{t+1} \cdot n^3)$ per layer. We introduce an optimization, called **SMT***, which is applicable when $(t+1)^2 \leq n$. It consists of replacing the set $\mathcal{Q}$ used within the Share, Handoff and Open protocols by some fixed set of $t+1$ pairwise disjoint subsets of $[n]$, each of size $t+1$. If at most t parties are corrupted, at least one of these is guaranteed to be fully honest, thus preserving security.

Corollary 13 (Corollary 3). *Assuming non-interactive homomorphic commitments, protocol **SMT*** realizes $\mathcal{F}_{\text{smt}}$ for d layers with polynomial communication $O(d \cdot n^{\frac{3}{2}})$, and is secure up to $t < \sqrt{n}$ corruptions per layer.*

9.1 Proactive Secret Sharing Scheme

We now describe the proactive secret sharing scheme, consisting on the sub-protocols **Share**, **Open** and **Handoff** for initially sharing the secret by the sender, reconstructing the secret in the end towards the receiver, and re-sharing the secret throughout intermediate layers (see Figures 11 to 13).

Notation. We write $[n] := \{1, \dots, n\}$. In this section and Appendix B, bold symbols such as $\mathbf{C}$ are vectors in $\mathbb{G}^n$ ($\mathbb{G}$ is the commitment space, see Definition 13). The i -th component of $\mathbf{C}$ is denoted by $\mathbf{C}[i]$. When we say $C, \mathbf{C}$ are on a degree- t polynomial, we mean that there exists a degree- t polynomial f with $f(0) = C$ and $f(i) = \mathbf{C}[i]$ for $i \in [n]$. We write $\mathcal{Q} := \binom{[n]}{t+1}$ (note that this applies to SMT, for **SMT*** we use $\mathcal{Q} := \{\{k(t+1) + 1, \dots, (k+1)(t+1)\} \mid k = 0, \dots, t\}$). Sometimes we write $I \subseteq \mathcal{L}$ for $I \in \mathcal{Q}$ to denote $\{P_i \in \mathcal{L} \mid i \in I\}$ or, analogously $P_i \in I$ instead of P_i with $i \in I$.

Sharing State. Before describing the protocol, let us introduce the sharing state that the parties maintain throughout all intermediate layers (in-between

the sender and receiver layers): Let $\mathcal{L}$ be such an intermediate layer. Every honest party $P_i \in \mathcal{L}$ holds a commitment C of the original secret from the sender, and a list of size $|\mathcal{Q}|$. For every $\ell \in \mathcal{Q}$, the ℓ -th value in the list is either $\perp$ or a tuple $(\ell, \mathbf{C}_i^\ell, s_i^\ell, r_i^\ell)$ where $s_i^\ell, r_i^\ell \in \mathbb{F}_p$ are two Shamir shares and $\mathbf{C}_i^\ell \in \mathbb{G}^n$ is a vector of commitments. We will refer to ℓ as the *label* of the tuple $(\ell, \mathbf{C}_i^\ell, s_i^\ell, r_i^\ell)$.

Invariants. The sharing state held by honest parties in an intermediate layer $\mathcal{L}$ satisfies two invariants, denoted **validSharing** and **consistentCommitments**. The first invariant is defined w.r.t. a commitment $C \in \mathbb{G}$, and states that every commitment vector held by an honest party is consistent with C :

consistentCommitments($\mathcal{L}, C$): Every honest $P_i \in \mathcal{L}$ has stored the commitment C , and for any tuple $(\ell, \mathbf{C}_i^\ell, \cdot, \cdot)$ appearing in the list of an honest $P_i \in \mathcal{L}$, we have that $C, \mathbf{C}_i^\ell$ lie on a polynomial of degree at most t .

The second invariant states that there is one label $\ell \in \mathcal{Q}$ for which all honest $P_i \in \mathcal{L}$ hold the same commitment vector $\mathbf{C}_i^\ell$ and corresponding shares:

validSharing($\mathcal{L}$): For any set $H \subseteq \mathcal{L}$ of honest parties, there exists a label $\ell \in \mathcal{Q}$ and $\mathbf{C}^\ell \in \mathbb{G}^n$ such that every $P_i \in H$ has stored a tuple $(\ell, \mathbf{C}^\ell, s_i^\ell, r_i^\ell)$ where (s_i^ℓ, r_i^ℓ) is an opening to $\mathbf{C}^\ell[i]$.

Protocol Descriptions. The protocol **Share** (Figure 11) simply lets the sender with secret input s compute a Shamir-sharing $(s_1, \dots, s_n)$ of s as well as a Shamir-sharing $(r_1, \dots, r_n)$ of a random value r . The sender computes a commitment C to s as well as commitments C_i to each share s_i of s , using randomness r resp. using the share r_i of r . To each $P_i \in \mathcal{L}_1$, the sender sends all the computed commitments and the private opening (s_i, r_i) . Then, P_i stores the commitment C , and for each label $\ell \in \mathcal{Q}$, a separate tuple $(\ell, \mathbf{C}_i^\ell, s_i, r_i)$, where $\mathbf{C}_i^\ell = (C_1, \dots, C_n)$. Note that at this point, for an honest sender, all tuples held by a given party contain the same vector of commitments and the same shares, but this might not be the case after the invocations of the **Handoff** protocol.

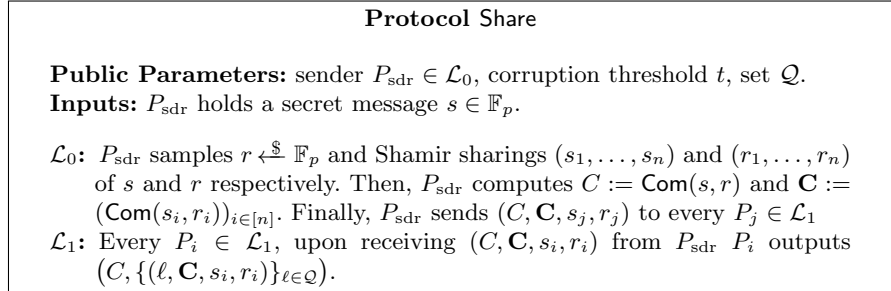


Fig. 11. Sharing Protocol

In protocol **Open** (Figure 12), every party $P_i \in \mathcal{L}_0$ (we denote the starting layer $\mathcal{L}_0$) holds a commitment C to the original sender's secret, and for each label $\ell \in \mathcal{Q}$, a separate tuple $(\ell, \mathbf{C}_i^\ell, s_i^\ell, r_i^\ell)$. Even after the intermediate **Handoff** steps, the tuples will satisfy the two invariants outlined above if P_{sdr} was honest.

Each $P_i \in \mathcal{L}_0$ simply sends C and all tuples $(\ell, \mathbf{C}_i^\ell, s_i^\ell, r_i^\ell)$ to the receiver $P_{\text{rcv}} \in \mathcal{L}_1$. The receiver checks if, for some label $\ell \in \mathcal{Q}$, there exists $\mathbf{C}^\ell \in \mathbb{G}^n$ such that C and $\mathbf{C}^\ell$ lie on a degree- t polynomial, and enough P_i sent shares s_i, r_i which form a correct opening of the i -th component of $\mathbf{C}^\ell$. If $t + 1$ such shares were received, P_{rcv} is able to reconstruct an opening of C .

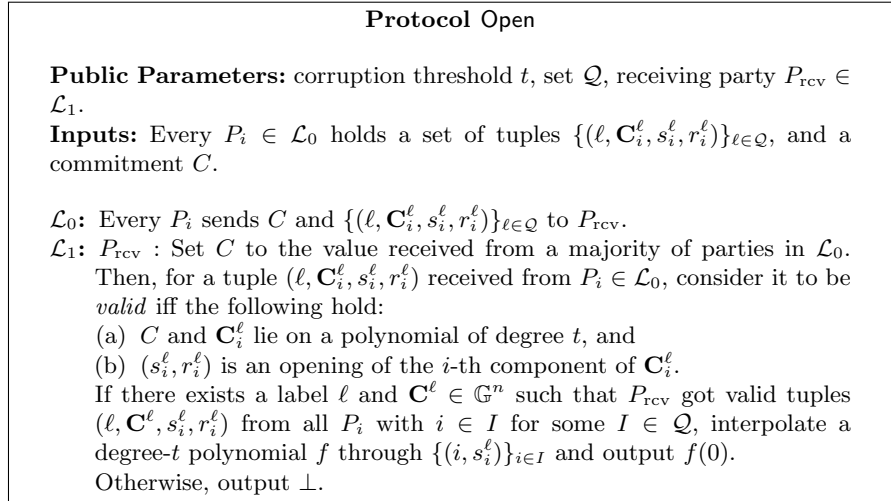


Fig. 12. Open protocol

In protocol **Handoff** (Figure 13), every party $P_i \in \mathcal{L}_0$ holds a commitment C and, for each label $\ell \in \mathcal{Q}$, a separate tuple $(\ell, \mathbf{C}_i^\ell, s_i^\ell, r_i^\ell)$. Then, for every $\ell \in \mathcal{Q}$, every P_i will distribute Shamir sharings of s_i^ℓ, r_i^ℓ among $\mathcal{L}_1$, together with commitments to the new subshares of s_i^ℓ (using the shares of r_i^ℓ as randomness). Parties in $\mathcal{L}_1$ iterate through all subsets $I \in \mathcal{Q}$, and use the commitments to check if there is a sharing which all parties in I handed off. Then, they select just one of these (the one with maximum label ℓ), and use the received subshares and commitments to subshares to interpolate new shares, as well as a vector of commitments to all new shares, which they will store under the label I . Essentially, this amounts to performing the classical resharing technique, which may fail if some parties in I are corrupt. However, one of the $I \in \mathcal{Q}$ is guaranteed to be fully honest. Thus, we can show that if the previous sharing state was valid (i.e. fulfilled the two invariants stated above), the same holds for the new sharing state. See Appendix B for proofs.

Protocol Handoff

Public: threshold t , set $\mathcal{Q} = \{S \subset [n] : |S| = t + 1\}$.

Input of each old party $P_i \in \mathcal{L}_{\text{old}}$: set of tuples $\{(\ell, \mathbf{C}_i^\ell, s_i^\ell, r_i^\ell)\}_{\ell \in \mathcal{Q}}$, commitment C to the secret.

Old party $P_i \in \mathcal{L}_{\text{old}}$:

Send C to each $P_j \in \mathcal{L}_{\text{new}}$. Further, for each tuple $(\ell, \mathbf{C}_i^\ell, s_i^\ell, r_i^\ell) \neq \perp$:

1. Sample degree- t Shamir sharings $(s_{i,1}^\ell, \dots, s_{i,n}^\ell) \leftarrow \text{Share}(s_i^\ell)$ and $(r_{i,1}^\ell, \dots, r_{i,n}^\ell) \leftarrow \text{Share}(r_i^\ell)$.
2. For all $j \in [n]$, set $\tilde{\mathbf{C}}_i^\ell[j] \leftarrow \text{Com}(s_{i,j}^\ell, r_{i,j}^\ell)$.
3. Send to each $P_j \in \mathcal{L}_{\text{new}}$:

$$(\ell, \tilde{\mathbf{C}}_i^\ell, s_{i,j}^\ell, r_{i,j}^\ell)$$

New party $P_j \in \mathcal{L}_{\text{new}}$:

1. Set C to the value received from a majority of parties in $\mathcal{L}_{\text{old}}$.
2. For each $I \in \mathcal{Q}$:
 - (a) Let ℓ^* be the maximum label such that all the following hold:
 - i. Every $P_i \in I$ sent a tuple $(\ell^*, \tilde{\mathbf{C}}_i^{\ell^*}, s_{i,j}^{\ell^*}, r_{i,j}^{\ell^*})$
 - ii. For every $i \in I$: $\tilde{\mathbf{C}}_i^{\ell^*}[j] = \text{Com}(s_{i,j}^{\ell^*}, r_{i,j}^{\ell^*})$
 - iii. For every $i \in I$, there exists $C_i^* \in \mathbb{G}$ such that $C_i^*, \tilde{\mathbf{C}}_i^{\ell^*}$ lie on a degree- t polynomial.
 - iv. The points $(0, C), \{(i, C_i^*)\}_{i \in I}$ lie on a degree- t polynomial.

If no such ℓ^* exists, store $\perp$ and proceed to the next set.
 - (b) Let $f_j^I(x)$ (resp. $g_j^I(x)$) be the degree- t polynomial going through the points $\{(i, s_{i,j}^{\ell^*})\}_{i \in I}$ (resp. $\{(i, r_{i,j}^{\ell^*})\}_{i \in I}$). Let $h_j^I(x)$ be the degree- t polynomial going through the points $\{(i, \tilde{\mathbf{C}}_i^{\ell^*})\}_{i \in I}$.
 - (c) Compute
$$(s_j^{\text{new}, I}, r_j^{\text{new}, I}) \leftarrow (f_j^I(0), g_j^I(0))$$

$$\mathbf{C}_j^{\text{new}, I}[k] \leftarrow h_j^I(0) \text{ (for all } k \in [n])$$
 - (d) Store $(I, \mathbf{C}_j^{\text{new}, I}, s_j^{\text{new}, I}, r_j^{\text{new}, I})$ for I .

Fig. 13. Re-sharing from old committee $\mathcal{L}_{\text{old}}$ to new committee $\mathcal{L}_{\text{new}}$.

10 Conclusions

We gave a full characterization on the feasibility of broadcast and consensus in the layered setting. Our protocols can be directly plugged into prior MPC protocols with dynamic committees to achieve the results without the need for ideal broadcast channels. In particular, a complete characterization of YOSO MPC without assuming broadcast channels.

In addition, we construct secure message transmission (SMT) protocols for the layered MPC model which do not assume broadcast or PKI, and which have linear communication complexity in the number of intermediate layers. Using homomorphic commitments, we propose a secure message transmission protocol for $t < n/2$ (with communication complexity exponential in n) and one for $t < \sqrt{n}$ (with communication complexity polynomial in n). Using the latter SMT, we obtain the first polynomial Layered MPC protocol with guaranteed output and without setup for the same threshold. Our work leaves open the question of whether polynomial-communication Layered MPC with guaranteed output delivery is possible without setup nor broadcast for the optimal $t < n/3$.

References

1. Abraham, I., Chan, T.-H.H., Dolev, D., Nayak, K., Pass, R., Ren, L., Shi, E.: Communication Complexity of Byzantine Agreement, Revisited. Robinson, P., Ellen, F. (eds.) 38th ACM PODC, pp. 317–326. ACM (2019). <https://doi.org/10.1145/3293611.3331629>
2. Acharya, A., Hazay, C., Kolesnikov, V., Prabhakaran, M.: Malicious Security for SCALES - Outsourced Computation with Ephemeral Servers. Reyzin, L., Stebila, D. (eds.) Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part IX. LNCS, vol. 14928, pp. 3–38. Springer, Heidelberg (2024). https://doi.org/10.1007/978-u3-u031-u68400-u5_1
3. Acharya, A., Hazay, C., Kolesnikov, V., Prabhakaran, M.: SCALES - MPC with Small Clients and Larger Ephemeral Servers. Kiltz, E., Vaikuntanathan, V. (eds.) TCC 2022, Part II. LNCS, pp. 502–531. Springer, Cham (2022). https://doi.org/10.1007/978-u3-u031-u22365-u5_18
4. Acharya, A., Hazay, C., Kolesnikov, V., Prabhakaran, M.: Towards Building Efficient SCALES Protocols. IACR Cryptol. ePrint Arch. (2025). <https://eprint.iacr.org/2025/1487>
5. Ben-Or, M., Goldwasser, S., Wigderson, A.: Completeness Theorems for Non-Cryptographic Fault-Tolerant Distributed Computation (Extended Abstract). 20th ACM STOC, pp. 1–10. ACM Press (1988). <https://doi.org/10.1145/62212.62213>
6. Benhamouda, F., Gentry, C., Gorbunov, S., Halevi, S., Krawczyk, H., Lin, C., Rabin, T., Reyzin, L.: Can a Public Blockchain Keep a Secret? Pass, R., Pietrzak, K. (eds.) Theory of Cryptography - 18th International Conference, TCC 2020, Durham, NC, USA, November 16-19, 2020, Proceedings, Part I. LNCS, vol. 12550, pp. 260–290. Springer, Heidelberg (2020). https://doi.org/10.1007/978-u3-u030-u64375-u1_10
7. Berman, P., Garay, J.A., Perry, K.J.: “Bit Optimal Distributed Consensus”. Computer Science: Research and Applications. Ed. by R. Baeza-Yates and U. Manber. Boston, MA: Springer US, 1992, pp. 313–321. ISBN: 978-1-4615-3422-8. https://doi.org/10.1007/978-u1-u4615-u3422-u8_27
8. Bhadauria, R., Chiang, J.H., Ravi, D., Sternad, J., Yakubov, S.: Minimizing the Use of the Honest Majority in YOSO MPC with Guaranteed Output Delivery. IACR Cryptol. ePrint Arch. (2024). <https://eprint.iacr.org/2024/2059>

9. Bienstock, A., Escudero, D., Polychroniadou, A.: On Linear Communication Complexity for (Maximally) Fluid MPC. Handschuh, H., Lysyanskaya, A. (eds.) CRYPTO 2023, Part I. LNCS, pp. 263–294. Springer, Cham (2023). https://doi.org/10.1007/978-u3-u031-u38557-u5_9
10. Bienstock, A., Escudero, D., Polychroniadou, A.: Perfectly Secure Fluid MPC with Abort and Linear Communication Complexity. IACR Commun. Cryptol. **1**(4), 28 (2024). <https://doi.org/10.62056/AESG89N4E>
11. Blum, E., Katz, J., Liu-Zhang, C.-D., Loss, J.: Asynchronous Byzantine Agreement with Subquadratic Communication. Pass, R., Pietrzak, K. (eds.) TCC 2020, Part I. LNCS, pp. 353–380. Springer, Cham (2020). https://doi.org/10.1007/978-u3-u030-u64375-u1_13
12. Boneh, D., Lynn, B., Shacham, H.: Short signatures from the Weil pairing. International conference on the theory and application of cryptology and information security, pp. 514–532 (2001)
13. Boyle, E., Cohen, R., Goel, A.: Breaking the $O(\sqrt{n})$ -Bit Barrier: Byzantine Agreement with Polylog Bits Per Party. Miller, A., Censor-Hillel, K., Korhonen, J.H. (eds.) 40th ACM PODC, pp. 319–330. ACM (2021). <https://doi.org/10.1145/3465084.3467897>
14. Braun, L., Damgård, I., Orlandi, C.: Secure Multiparty Computation from Threshold Encryption Based on Class Groups. Handschuh, H., Lysyanskaya, A. (eds.) CRYPTO 2023, Part I. LNCS, pp. 613–645. Springer, Cham (2023). https://doi.org/10.1007/978-u3-u031-u38557-u5_20
15. Campanelli, M., David, B., Khoshakhlagh, H., Konring, A., Nielsen, J.B.: Encryption to the Future - A Paradigm for Sending Secret Messages to Future (Anonymous) Committees. Agrawal, S., Lin, D. (eds.) ASIACRYPT 2022, Part III. LNCS, pp. 151–180. Springer, Cham (2022). https://doi.org/10.1007/978-u3-u031-u22969-u5_6
16. Canetti, R.: Security and Composition of Multiparty Cryptographic Protocols. Journal of Cryptology **13**(1), 143–202 (2000). <https://doi.org/10.1007/s001459910006>
17. Canetti, R.: Universally Composable Security: A New Paradigm for Cryptographic Protocols. 42nd FOCS, pp. 136–145. IEEE Computer Society Press (2001). <https://doi.org/10.1109/SFCS.2001.959888>
18. Canetti, R., Kolby, S., Ravi, D., Soria-Vazquez, E., Yakoubov, S.: Taming Adaptivity in YOSO Protocols: The Modular Way. Theory of Cryptography (2023)
19. Cascudo, I., David, B., Garms, L., Konring, A.: YOLO YOSO: Fast and Simple Encryption and Secret Sharing in the YOSO Model. Agrawal, S., Lin, D. (eds.) ASIACRYPT 2022, Part I. LNCS, pp. 651–680. Springer, Cham (2022). https://doi.org/10.1007/978-u3-u031-u22963-u3_22
20. Chaum, D., Crépeau, C., Damgård, I.: Multiparty Unconditionally Secure Protocols (Abstract) (Informal Contribution). Pomerance, C. (ed.) CRYPTO’87. LNCS, p. 462. Springer, Berlin, Heidelberg (1988). https://doi.org/10.1007/3-u540-u48184-u2_43
21. Chen, J., Micali, S.: Algorand: A secure and efficient distributed ledger. Theoretical Computer Science **777**, 155–183 (2019)
22. Choudhuri, A.R., Goel, A., Green, M., Jain, A., Kaptchuk, G.: Fluid MPC: Secure Multiparty Computation with Dynamic Participants. Malkin, T., Peikert, C. (eds.) CRYPTO 2021, Part II. LNCS, pp. 94–123. Springer, Cham, Virtual Event (2021). https://doi.org/10.1007/978-u3-u030-u84245-u1_4

23. Damgård, I.B., Kamp, S.H., Loss, J., Nielsen, J.B.: Asynchronous YOSO a la Paillier. *IACR Cryptol. ePrint Arch.* (2025)
24. David, B., Deligios, G., Goel, A., Ishai, Y., Konring, A., Kushilevitz, E., Liu-Zhang, C.-D., Narayanan, V.: Perfect MPC over Layered Graphs. Handschuh, H., Lysyanskaya, A. (eds.) *CRYPTO 2023, Part I. LNCS*, pp. 360–392. Springer, Cham (2023). https://doi.org/10.1007/978-u3-u031-u38557-u5_12
25. Deligios, G., Goel, A., Liu-Zhang, C.-D.: Maximally-Fluid MPC with Guaranteed Output Delivery, *Cryptology ePrint Archive*, Paper 2023/415 (2023). <https://eprint.iacr.org/2023/415>.
26. Deligios, G., Konring, A., Liu-Zhang, C.-D., Narayanan, V.: Statistical Layered MPC. Boyle, E., Mahmood, M. (eds.) *TCC 2024, Part IV. LNCS*, pp. 362–394. Springer, Cham (2024). https://doi.org/10.1007/978-u3-u031-u78023-u3_12
27. Desmedt, Y., Jajodia, S.: Redistributing secret shares to new access structures and its applications, (1997). Tech Report, George Mason U.
28. Dolev, D., Strong, H.R.: Authenticated algorithms for Byzantine agreement. *SIAM Journal on Computing* **12**(4), 656–666 (1983)
29. ElGamal, T.: A Public Key Cryptosystem and a Signature Scheme Based on Discrete Logarithms. *IEEE Transactions on Information Theory* **31**(4), 469–472 (1985). <https://doi.org/10.1109/TIT.1985.1057074>
30. Escudero, D., Masserova, E., Polychroniadou, A.: Brief Announcement: Towards Scalable YOSO MPC via Packed Secret-Sharing. Balliu, A., Kuhn, F. (eds.) *Proceedings of the ACM Symposium on Principles of Distributed Computing, PODC 2025, Hotel Las Brisas Huatulco, Huatulco, Mexico, June 16-20, 2025*, pp. 162–165. ACM (2025). <https://doi.org/10.1145/3732772.3733556>
31. Escudero, D., Masserova, E., Polychroniadou, A.: Towards Scalable YOSO MPC via Packed Secret-Sharing. *IACR Cryptol. ePrint Arch.* (2025). <https://eprint.iacr.org/2025/635>
32. Feldman, P., Micali, S.: An optimal probabilistic protocol for synchronous Byzantine agreement. *SIAM Journal on Computing* **26**(4), 873–933 (1997)
33. Fitzi, M., Liu-Zhang, C.-D., Loss, J.: A New Way to Achieve Round-Efficient Byzantine Agreement. Miller, A., Censor-Hillel, K., Korhonen, J.H. (eds.) *40th ACM PODC*, pp. 355–362. ACM (2021). <https://doi.org/10.1145/3465084.3467907>
34. Garay, J.A.: Reaching (and Maintaining) Agreement in the Presence of Mobile Faults (Extended Abstract). Tel, G., Vitányi, P.M.B. (eds.) *Distributed Algorithms, 8th International Workshop, WDAG '94, Terschelling, The Netherlands, September 29 - October 1, 1994, Proceedings. LNCS*, vol. 857, pp. 253–264. Springer, Heidelberg (1994). <https://doi.org/10.1007/BFB0020438>
35. Gennaro, R., Rabin, M.O., Rabin, T.: Simplified VSS and Fast-Track Multiparty Computations with Applications to Threshold Cryptography. *PODC* (1998)
36. Gentry, C., Halevi, S., Krawczyk, H., Magri, B., Nielsen, J.B., Rabin, T., Yakubov, S.: YOSO: You Only Speak Once - Secure MPC with Stateless Ephemeral Roles. Malkin, T., Peikert, C. (eds.) *CRYPTO 2021, Part II. LNCS*, pp. 64–93. Springer, Cham, Virtual Event (2021). https://doi.org/10.1007/978-u3-u030-u84245-u1_3
37. Goldreich, O., Micali, S., Wigderson, A.: How to Play any Mental Game or A Completeness Theorem for Protocols with Honest Majority. Aho, A. (ed.) *19th ACM STOC*, pp. 218–229. ACM Press (1987). <https://doi.org/10.1145/28395.28420>

38. Goldwasser, S., Micali, S., Rivest, R.L.: A digital signature scheme secure against adaptive chosen-message attacks. *SIAM Journal on computing* **17**(2), 281–308 (1988)
39. Goyal, V., Kothapalli, A., Masserova, E., Parno, B., Song, Y.: Storing and Retrieving Secrets on a Blockchain. Hanaoka, G., Shikata, J., Watanabe, Y. (eds.) *Public-Key Cryptography - PKC 2022 - 25th IACR International Conference on Practice and Theory of Public-Key Cryptography*, Virtual Event, March 8–11, 2022, Proceedings, Part I. LNCS, vol. 13177, pp. 252–282. Springer, Heidelberg (2022). <https://doi.org/10.1007/978-u3-u030-u97121-u2\10>
40. Goyal, V., Masserova, E., Parno, B., Song, Y.: Blockchains Enable Non-interactive MPC. Nissim, K., Waters, B. (eds.) *TCC 2021, Part II*. LNCS, pp. 162–193. Springer, Cham (2021). https://doi.org/10.1007/978-u3-u030-u90453-u1_6
41. Guo, Y., Pass, R., Shi, E.: Synchronous, with a Chance of Partition Tolerance. Boldyreva, A., Micciancio, D. (eds.) *CRYPTO 2019, Part I*. LNCS, pp. 499–529. Springer, Cham (2019). https://doi.org/10.1007/978-u3-u030-u26948-u7_18
42. Hirt, M., Maurer, U.M.: Player Simulation and General Adversary Structures in Perfect Multiparty Computation. *Journal of Cryptology* **13**(1), 31–60 (2000). <https://doi.org/10.1007/s001459910003>
43. Johnson, D., Menezes, A., Vanstone, S.: The elliptic curve digital signature algorithm (ECDSA). *International journal of information security* **1**, 36–63 (2001)
44. Katz, J., Koo, C.-Y.: On Expected Constant-Round Protocols for Byzantine Agreement. Dwork, C. (ed.) *CRYPTO 2006*. LNCS, pp. 445–462. Springer, Berlin, Heidelberg (2006). https://doi.org/10.1007/11818175_27
45. Kolby, S., Ravi, D., Yakubov, S.: Constant-Round YOSO MPC Without Setup. *IACR Commun. Cryptol.* **1**(3), 30 (2024). <https://doi.org/10.62056/AE5W4FE-u3>
46. Lamport, L., Fischer, M.: Byzantine generals and transaction commit protocols. Unpublished (1982)
47. Liu-Zhang, C.-D., Masserova, E., Ribeiro, J., Soni, P., Thyagarajan, S.A.K.: Efficient Distributed Randomness Generation from Minimal Assumptions Where Parties Speak Sequentially Once. Fehr, S., Fouque, P.-A. (eds.) *EUROCRYPT 2025, Part V*. LNCS, pp. 176–206. Springer, Cham (2025). https://doi.org/10.1007/978-u3-u031-u91092-u0_7
48. Liu-Zhang, C.-D., Masserova, E., Ribeiro, J., Soni, P., Thyagarajan, S.A.K.: Improved YOSO Randomness Generation with Worst-Case Corruptions. Clark, J., Shi, E. (eds.) *FC 2024, Part II*. LNCS, pp. 73–89. Springer, Cham (2024). https://doi.org/10.1007/978-u3-u031-u78679-u2_4
49. Micali, S., Vaikuntanathan, V.: Optimal and player-replaceable consensus with an honest majority. (2017)
50. Nielsen, J.B., Ribeiro, J.L., Obremski, M.: Public Randomness Extraction with Ephemeral Roles and Worst-Case Corruptions. Dodis, Y., Shrimpton, T. (eds.) *CRYPTO 2022, Part I*. LNCS, pp. 127–147. Springer, Cham (2022). https://doi.org/10.1007/978-u3-u031-u15802-u5_5
51. Pass, R., Shi, E.: The Sleepy Model of Consensus. Takagi, T., Peyrin, T. (eds.) *ASIACRYPT 2017, Part II*. LNCS, pp. 380–409. Springer, Cham (2017). https://doi.org/10.1007/978-u3-u319-u70697-u9_14
52. Pedersen, T.P.: Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing. Feigenbaum, J. (ed.) *CRYPTO’91*. LNCS, pp. 129–140. Springer, Berlin, Heidelberg (1992). https://doi.org/10.1007/3-u540-u46766-u1_9

53. Rabin, T., Ben-Or, M.: Verifiable Secret Sharing and Multiparty Protocols with Honest Majority (Extended Abstract). 21st ACM STOC, pp. 73–85. ACM Press (1989). <https://doi.org/10.1145/73007.73014>
54. Rachuri, R., Scholl, P.: Le Mans: Dynamic and Fluid MPC for Dishonest Majority. Dodis, Y., Shrimpton, T. (eds.) CRYPTO 2022, Part I. LNCS, pp. 719–749. Springer, Cham (2022). https://doi.org/10.1007/978-u3-u031-u15802-u5_25
55. Schnorr, C.-P.: Efficient identification and signatures for smart cards. Advances in Cryptology—CRYPTO’89 Proceedings 9, pp. 239–252 (1990)
56. Yao, A.C.-C.: How to Generate and Exchange Secrets (Extended Abstract). 27th FOCS, pp. 162–167. IEEE Computer Society Press (1986). <https://doi.org/10.1109/SFCS.1986.25>

A Proofs regarding Weak and Graded Consensus

We restate the theorems and show the missing proofs for the protocols from section 4.

Theorem 14 (Theorem 5). *Protocol WC defined in Figure 2 is an $(n, t, 1)$ -layered weak consensus protocol for any $t < \frac{n}{3}$.*

Proof. First, note that if all honest parties in $\mathcal{L}_0$ hold the same value x , then, as there are at least $n - t$ honest parties in $\mathcal{L}_0$, every party in $\mathcal{L}_1$ receives x from at least $n - t$ parties. Additionally, in this case at most t parties in $\mathcal{L}_0$ send values different from x . As $t < \frac{n}{3}$, we get $t < n - t$. Thus, every honest party in $\mathcal{L}_1$ outputs x . Hence, validity holds.

Second, if an honest party in $\mathcal{L}_1$ outputs a value $y \neq \perp$, then it received y from at least $n - t$ parties of $\mathcal{L}_0$, hence in particular at least $n - 2t$ honest parties sent y . Thus, there are at most $n - (n - 2t) = 2t$ parties in $\mathcal{L}_0$ that could send some $y' \neq y$, and as $2t < n - t$ for $t < \frac{n}{3}$, we get that every honest party in $\mathcal{L}_1$ outputs y or $\perp$. Hence, weak consistency holds. $\square$

Theorem 15 (Theorem 6). *Protocol GC defined in Figure 3 is an $(n, t, 2)$ -layered graded consensus protocol for any $t < \frac{n}{3}$.*

Proof. First, if all honest parties in $\mathcal{L}_0$ hold the same value x , then, as there are at least $n - t$ honest parties in $\mathcal{L}_0$, every party in $\mathcal{L}_1$ receives x from at least $n - t$ parties, and at most t parties in $\mathcal{L}_0$ send values different from x . By the protocol description, every honest party in $\mathcal{L}_1$ then forwards x to everyone in $\mathcal{L}_2$. As there are at least $n - t$ honest parties in $\mathcal{L}_1$, every honest $P_i \in \mathcal{L}_2$ outputs $(x, 2)$. Thus, validity holds.

Second, note that if at least one honest party in $\mathcal{L}_1$ sends a message x to all parties in $\mathcal{L}_2$, then no honest party $\mathcal{L}_1$ sends a message $x' \neq x$ to anyone in layer $\mathcal{L}_2$ (otherwise, since $(n - 2t) + (n - 2t) - (n - t) = n - 3t > 0$, at least one honest party in $\mathcal{L}_0$ sent two different messages). $(\dagger)$

Next, if some honest party in layer $\mathcal{L}_2$ outputs (y, g) with $g \geq 1$, then it received y from at least $t + 1$ parties of $\mathcal{L}_1$, hence in particular at least one honest party in $\mathcal{L}_1$ sent y . By $(\dagger)$, we get that there is no message $y' \neq y$ that

an honest party from $\mathcal{L}_1$ sends to parties in layer $\mathcal{L}_2$. Thus, every honest party in $\mathcal{L}_2$ receives at most t messages with some $y' \neq y$, hence they never output (y', g) for $y' \neq y$ and $g \geq 1$.

Moreover, if some honest party in layer $\mathcal{L}_2$ outputs $(y, 2)$, then it received y from at least $n - t$ parties of $\mathcal{L}_1$, hence in particular at least $n - 2t$ honest parties in $\mathcal{L}_1$ sent y . As $t < \frac{n}{3}$, we get $n - 2t > t$, hence every P_i in layer $\mathcal{L}_2$ received y from at least $t + 1$ parties. Thus, the protocol satisfies graded consistency. $\square$

Theorem 16 (Theorem 7). *Assuming a plain PKI, protocol aWC defined in Figure 4 is an $(n, t, 2)$ -layered weak consensus protocol for any $t < \frac{n}{2}$.*

Proof. First, note that if all honest parties in $\mathcal{L}_0$ hold the same value x , every party in $\mathcal{L}_1$ receives correct signatures on x from at least $n - t$ parties. Thus, every honest party in $\mathcal{L}_1$ sends a degree-0-certificate for x to everyone in $\mathcal{L}_2$, hence every party in $\mathcal{L}_2$ receives at least $n - t$ degree-0-certificates for x . Furthermore, at most t parties in $\mathcal{L}_0$ send values different from x . As $t < \frac{n}{2}$, we get $t < n - t$. Thus there cannot be a degree-0-certificate for $1 - x$. Thus, every honest party in $\mathcal{L}_2$ outputs x and validity holds.

Second, if an honest party in $\mathcal{L}_2$ outputs a value $y \neq \perp$, then it received $n - t$ degree-0-certificates for y . As $t < n - t$, at least one of these was sent by an honest party to all parties in $\mathcal{L}_2$. Therefore, no honest party outputs $(1 - y)$, which proves weak consistency. $\square$

Theorem 17 (Theorem 8). *Assuming a plain PKI, protocol aGC defined in Figure 5 is an $(n, t, 4)$ -layered graded consensus protocol for any $t < \frac{n}{2}$.*

Proof. We first prove validity. Assume that all honest parties in $\mathcal{L}_0$ hold the same bit b . Since $t < n - t$, there are not enough signatures by parties in $\mathcal{L}_0$ on $(1 - b)$ to form a degree-0-certificate, which implies no honest party ever receives a certificate of any degree for $(1 - b)$. Moreover, every honest $P_i \in \mathcal{L}_1$ receives $n - t$ signatures on b and sends a degree-0-certificate for b to in $\mathcal{L}_2$. By analogous reasoning, all honest parties in $\mathcal{L}_2, \mathcal{L}_3$ and $\mathcal{L}_4$ receive at least $n - t$ certificates for b of degree 0, 1, and 2 respectively. Thus, all honest parties in $\mathcal{L}_4$ output $(b, 2)$.

We now prove graded consistency. For the first case, assume that one honest party $P_i \in \mathcal{L}_4$ outputs $(b, 2)$. This means that P_i received degree-2-certificates for b from $n - t$ parties in $\mathcal{L}_3$, at least one of which was honest. This implies that all parties in $\mathcal{L}_4$ received a degree-2-certificate for b . Moreover, since an honest party in $\mathcal{L}_3$ sent out a degree-2-certificate for b , it follows that no honest party in $\mathcal{L}_2$ sent a degree-1-certificate for $(1 - b)$ and thus it is impossible for any party in $\mathcal{L}_3$ to form a degree-2-certificate for $(1 - b)$. Therefore, we have that every honest party in $\mathcal{L}_4$ outputs $(b, 1)$ or $(b, 2)$.

We move on to the second case: assume some honest $P_i \in \mathcal{L}_4$ outputs $(b, 1)$. Since P_i received a degree-2-certificate for b , there are at least $n - t$ parties in $\mathcal{L}_2$ (at least one of which is honest) who signed a degree-1-certificate for b . Thus,

no honest party in $\mathcal{L}_1$ signed a degree-0-certificate for $(1-b)$, which means it is impossible for any party in $\mathcal{L}_3$ to form a degree-2-certificate for $(1-b)$. Therefore, no honest party in $\mathcal{L}_4$ outputs $(1-b, g)$ with $g \geq 1$. $\square$

Proposition 2 (Proposition 1). *Assume $t < n/2$. Let π be an (n, t, d) -layered gradedcast protocol and $d' > d$. Then there exists an (n, t, d') -layered protocol π' which achieves gradedcast towards $\mathcal{L}_d \cup \mathcal{L}_{d'}$, with graded consistency holding for any pair of honest parties $P, P' \in \mathcal{L}_d \cup \mathcal{L}_{d'}$.*

Proof. We construct such π' as follows: First, parties in $\mathcal{L}_0, \dots, \mathcal{L}_d$ execute π . Then, every party in $\mathcal{L}_d$ sends their output from π to all parties in $\mathcal{L}_{d'}$. Parties in $\mathcal{L}_{d'}$ sort all received values using the ordering $(0, 2) < (0, 1) < (\perp, 0) < (1, 1) < (1, 2)$. Finally, parties in $\mathcal{L}_{d'}$ output the median of the resulting list.

This preserves validity: If P_{sdr} is honest with input x , all honest parties in $\mathcal{L}_d$ will output $(x, 2)$ by validity of π . Parties in $\mathcal{L}_{d'}$ will receive $(x, 2)$ from at least $n - t > n/2$ parties in $\mathcal{L}_d$, so the median value they compute is also $(x, 2)$. Next, we show graded consistency, i.e., that the outputs of all honest parties are contained in a set of two consecutive values (w.r.t. the aforementioned ordering). Again, it follows using $t < n/2$ that if the outputs of all honest parties in $\mathcal{L}_d$ fulfil this property, then the outputs of all honest parties in $\mathcal{L}_{d'}$ are contained in the same set of two consecutive values. $\square$

B Proofs regarding SMT

B.1 Extended Preliminaries

We first recap some building blocks used within SMT. In the following, $\mathbb{F}_p$ denotes some prime field.

Definition 12 (Shamir Sharing). *Given any $s \in \mathbb{F}_p$, we write $(s_1, \dots, s_n) \leftarrow \text{Share}(s)$ to denote a Shamir sharing of s with degree t , i.e. $(s_1, \dots, s_n)$ is obtained by sampling uniformly random $a_1, \dots, a_t \in \mathbb{F}_p$ and computing $s_i := f(i)$, where $f(x) := s + a_1x + \dots + a_tx^t$.*

Note that Shamir sharing satisfies 1) t -privacy: Any t shares reveal nothing about the secret, 2) $(t+1)$ -reconstruction: From any $t+1$ shares, one can recover the secret, and 3) linearity: For any $s, s', \mu \in \mathbb{F}_p$, we have that $\text{Share}(s) + \mu \cdot \text{Share}(s')$ is a degree- t sharing of $s + \mu s'$.

Definition 13 (Commitments). *We define a **non-interactive commitment scheme** with message space $\mathcal{M}$, randomness space $\mathcal{R}$, and commitment space $\mathcal{C}$ to be a triple of PPT algorithms $(\text{Setup}, \text{Com}, \text{Open})$ such that:*

- $\text{pp} \leftarrow \text{Setup}(1^\lambda)$: The setup algorithm takes as input the security parameter λ and produces a set of public parameters pp

- $\text{com} \leftarrow \text{Com}(\text{pp}, m, r)$: The commit algorithm takes as input the public parameters pp , a message $m \in \mathcal{M}$, and randomness $r \in \mathcal{R}$ and produces a commitment $\text{com} \in \mathcal{C}$;
- $b \leftarrow \text{Open}(\text{pp}, \text{com}, m, r)$: The open algorithm takes as input the public parameters pp , a commitment $\text{com} \in \mathcal{C}$, a message $m \in \mathcal{M}$, and randomness $r \in \mathcal{R}$ and outputs a bit $b \in \{0, 1\}$;

We say that such a commitment scheme is:

- **Perfectly Hiding** if for all $m' \neq m$ and for $\text{pp} \leftarrow \text{Setup}(1^\lambda)$ the distributions $\text{Com}(\text{pp}, m, r)$ and $\text{Com}(\text{pp}, m', r)$ are equal, where the distribution is over the randomness r ;
- **Computationally Binding** if any PPT adversary $\mathcal{A}$ has a negligible probability (in the security parameter λ) of outputting (m, r) and (m', r') such that $m \neq m'$ and $\text{Com}(\text{pp}, m, r) = \text{Com}(\text{pp}, m', r')$;
- **Linearly Homomorphic** if there exist a binary function $+: \mathcal{C}^2 \rightarrow \mathcal{C}$ and an element $0 \in \mathcal{C}$ such that $(\mathcal{C}, +, 0)$ is an abelian group, a binary function $+: \mathcal{M}^2 \rightarrow \mathcal{M}$ and an element $0 \in \mathcal{M}$ such that $(\mathcal{M}, +, 0)$ is an abelian group, and a binary function $+: \mathcal{R}^2 \rightarrow \mathcal{R}$ and an element $0 \in \mathcal{R}$ such that $(\mathcal{R}, +, 0)$ is an abelian group, and for all $m, m' \in \mathcal{M}$ and all $r, r' \in \mathcal{R}$ it holds that $\text{Com}(\text{pp}, m + m', r + r') = \text{Com}(\text{pp}, m, r) + \text{Com}(\text{pp}, m', r')$ for all $(\text{pp}, \tau) \leftarrow \text{Setup}(1^\lambda)$;

In our setting we will consider $\mathcal{M} = \mathcal{R} = \mathbb{F}_p$ and commitment space $\mathcal{C}$ an $\mathbb{F}_p$ -module $(\mathbb{G}, +, 0)$.

B.2 Proof of Theorem 13

Proposition 3 (Correctness). *The value output by the receiver P_{rcv} , if any, is equal to the secret s of the sender P_{sdr} .*

Proof of Proposition 3 (correctness) Proposition 3 follows from applying recursively the following Lemma 1 in all the pairs of consecutive layers $(\mathcal{L}_{\text{old}}, \mathcal{L}_{\text{new}})$ from the sender to the receiver.

Lemma 1. *Consider $C \in \mathbb{G}$ such that the invariant $\text{consistentCommitments}(\mathcal{L}_{\text{old}}, C)$ (presented in Section 9.1) holds. Then, after an execution of **Handoff** between $\mathcal{L}_{\text{old}}$ and $\mathcal{L}_{\text{new}}$, $\text{consistentCommitments}(\mathcal{L}_{\text{new}}, C)$ also holds if at most $t < n/2$ parties in $\mathcal{L}_{\text{old}}$ are corrupt.*

Proof. Assume $\text{consistentCommitments}(\mathcal{L}_{\text{old}}, C)$ holds. Since a majority of parties in $\mathcal{L}_{\text{old}}$ are honest, all honest parties in $\mathcal{L}_{\text{new}}$ find C to be the most received commitment. Additionally, the check performed by honest parties in $\mathcal{L}_{\text{new}}$ during steps 2((a))iii and 2((a))iv in protocol **Handoff** ensures (by linearity of Shamir sharings) that for all tuples $(\ell, \mathbf{C}^\ell, \cdot, \cdot) \neq \perp$ stored by honest parties in $\mathcal{L}_{\text{new}}$ it holds that $C, \mathbf{C}$ lie on a degree- t polynomial.

Thus, $\text{consistentCommitments}(\mathcal{L}_{\text{new}}, C)$ holds. $\square$

Proposition 4 (Liveness). *The receiver $P_{\text{rcv}} \in \mathcal{L}_d$ outputs a value at the end of the d -th round.*

Proof of Proposition 4 (Liveness). Proposition 4 follows from applying the following Lemma 2 in all the pairs of consecutive layers $(\mathcal{L}_{\text{old}}, \mathcal{L}_{\text{new}})$ from the sender to the receiver. In the proofs of Proposition 4 below, we consider as a unique public fixed data the commitment C to the secret which is passed unchanged from layer to layer (see Lemma 1).

Lemma 2. *Assume that $\text{validSharing}(\mathcal{L}_{\text{old}})$ (presented in Section 9.1) holds, that there exists a commitment C such that $\text{consistentCommitments}(\mathcal{L}_{\text{old}}, C)$ holds and that at most $t < n/2$ parties in $\mathcal{L}_{\text{old}}$ are corrupt. Then, after executing Handoff between $\mathcal{L}_{\text{old}}$ and $\mathcal{L}_{\text{new}}$, we have that $\text{validSharing}(\mathcal{L}_{\text{new}})$ holds.*

Proof. Fix $I \in \mathcal{Q}$ such that all $P_i \in I \subseteq \mathcal{L}_{\text{old}}$ are honest. By $\text{validSharing}(\mathcal{L}_{\text{old}})$, there exists a label ℓ and $\mathbf{C} \in \mathbb{G}^n$ such that all $P_i \in I$ have stored a tuple starting with $(\ell, \mathbf{C})$ in their list. By $\text{consistentCommitments}(\mathcal{L}_{\text{old}}, C)$ we have that $C, \mathbf{C}$ lie on a degree- t polynomial. This implies that, when iterating over the set I , every honest $P_j \in \mathcal{L}_{\text{new}}$ will store a tuple, as all the checks in step (a) will be fulfilled for ℓ .

Moreover, as honest parties in $\mathcal{L}_{\text{old}}$ send a subsharing for any given ℓ to either all or no parties in $\mathcal{L}_{\text{new}}$, and in the former case send the same vector of commitments to subshares (with one correct opening), it follows that any ℓ either passes the check for all or for no honest $P_j \in \mathcal{L}_{\text{new}}$ iterating over I .

Thus, all honest $P_j \in \mathcal{L}_{\text{new}}$ pick the same maximum label ℓ^* for I , and they compute the same new commitment vector $\mathbf{C}^{\text{new}, I}$. It then follows from the fact that checks 2((a))iii and 2((a))iv passed that $C, \mathbf{C}^{\text{new}, I}$ lie on a degree- t polynomial (using linearity of Shamir sharings); and it follows from check 2((a))ii (using linearity of commitments) that any honest $P_j \in \mathcal{L}_{\text{new}}$ will obtain an opening to $\mathbf{C}^{\text{new}, I}[j]$.

Therefore, $\text{validSharing}(\mathcal{L}_{\text{new}})$ holds. $\square$

Proof of Theorem 13. We now show the claim of Theorem 13, which is that for any PPT adversary $\mathcal{A}$ in the real world corrupting $t < n/2$ parties, there exists a simulator Sim in an ideal world (where parties have access to an ideal functionality $\mathcal{F}_{\text{smt}}$) such that the joint distribution of the adversary's view and the honest parties' inputs and outputs is statistically indistinguishable between the two worlds.

Proof. We describe the simulator Sim for an execution with an honest sender and an honest receiver (we make no guarantees if the sender or receiver is corrupted). The simulator Sim will simulate the view of the adversary $\mathcal{A}$ in a real protocol execution by interacting solely with the adversary (including the corrupt parties).

Simulating the sender P_{sdr} . Denote $F \subset [n]$ and $H \subset [n]$ the subsets of indices of the t corrupt parties and of the $n - t$ honest parties in $\mathcal{L}_1$. Generate a random commitment C . For each index $i \in F$ of a corrupt party $P_i \in \mathcal{L}_1$, generate uniformly at random (s_i, r_i) , set $C_i = \text{Com}(s_i, r_i)$. Interpolate the remaining $(C_i)_{i \in H}$ through the $t + 1$ points $\{0\} \cup F$ and corresponding values $C, (C_i)_{i \in F}$,

so that the full vector $(C_i)_{i \in H}$ is a vector of evaluations of a degree- t polynomial with coefficients in $\mathbb{G}$, i.e., in $\mathbb{G}[X]$, which evaluates in particular to C at 0. Make P_{sdr} send to each corrupted party $P_i \in \mathcal{L}_1$ ($i \in F$) the quintuple

$$(C, (C_i)_{i \in [n]}, s_i, r_i)$$

and to each simulated honest party $P_i \in \mathcal{L}_1$ ($i \in H$) the quintuple

$$(C, (C_i)_{i \in [n]}, \perp, \perp)$$

Simulating the honest parties of an old layer $\mathcal{L}_{\text{old}}$ during a Handoff to the next layer $\mathcal{L}_{\text{new}}$. Denote $F' \subset [n]$ and $H' \subset [n]$ the subsets of indices of the t corrupt parties and of the $n - t$ honest parties in $\mathcal{L}_{\text{new}}$. For each (simulated) old honest party P_i and for each $\ell \in \mathcal{Q}$ for which P_i stored a tuple $(\ell, \mathbf{C}_i^\ell, s_i^\ell, r_i^\ell) \neq \perp$, generate uniformly at random $(s_{i,j}^\ell, r_{i,j}^\ell)$ for every $j \in F'$ and set $C_{ij}^\ell = \text{Com}(s_{i,j}^\ell, r_{i,j}^\ell)$. Interpolate the remaining $(C_{ij}^\ell)_{j \in H'}$ through the $t + 1$ points $\{0\} \cup F'$ and corresponding values $\mathbf{C}_i^\ell[j], (C_{ij}^\ell)_{j \in F'}$, so that the full vector $(C_{ij}^\ell)_{j \in [n]}$ is a vector of evaluations of a degree- t polynomial with coefficients in $\mathbb{G}$, i.e., in $\mathbb{G}[X]$, which evaluates in particular to $\mathbf{C}_i^\ell[j]$ at 0. Make P_i send C to each corrupted party $P'_j \in \mathcal{L}_{\text{new}}$ ($j \in F'$) as well as the tuples

$$(\ell, (C_{ij}^\ell)_{j \in [n]}, s_{i,j}^\ell, r_{i,j}^\ell)$$

for every $\ell \in \mathcal{Q}$. Make P_i send to each simulated honest party $P'_j \in \mathcal{L}_{\text{new}}$ ($i \in H'$) the quintuples

$$(\ell, (C_{ij}^\ell)_{j \in [n]}, \perp, \perp)$$

for every $\ell \in \mathcal{Q}$.

Simulating (silently) the honest parties of a “new” layer $\mathcal{L}_{\text{new}}$ during a Handoff from the “old” previous layer $\mathcal{L}_{\text{old}}$. Each $P'_j \in \mathcal{L}_{\text{new}}$ follows the protocol up to two changes: The first change is that for each $I \in \mathcal{Q}$, the check in step 2((a))ii (fig. 13) is only performed for corrupt $P_i \in \mathcal{L}_{\text{old}}$, as simulated honest parties don't send any subshares, and the second change is that instead of computing new shares in step (c), simulated honest parties set $(s_j^{\text{new}, I}, r_j^{\text{new}, I}) \leftarrow (\perp, \perp)$.

Proof of indistinguishability with a real execution. We consider a series of hybrid games going from the view in the real execution, to the view in the simulated one.

We first define the games $\text{Game}^{\text{strip}, r}$ for $r \in [d]$ as follows: $\text{Game}^{\text{strip}, d}$ is the real execution. Then, for $r \in [d - 1]$: $\text{Game}^{\text{strip}, r}$ is defined recursively as $\text{Game}^{\text{strip}, r+1}$, with the change that in the Handoff between $\mathcal{L}_{r-1}$ ¹² and $\mathcal{L}_r$ the new parties $P'_j \in \mathcal{L}_{\text{new}}$ behave (silently) as in the first change (1) of the simulated execution. Namely, for each I , they only apply the check in step 2((a))ii to tuples sent by corrupt parties. Since the honest parties only send correct data, it follows

¹² by $\mathcal{L}_0$ we mean simply P_{sdr}

that removing the check produces the same outcome. Hence, the view of the adversary is unchanged.

We then define the game **Game^{rcv}** as follows. **Game^{rcv}** is defined as the real execution, with the difference that the opening is simulated as done by Sim. Since the number of 5-uples sent by each simulated $P_i \in \mathcal{L}_{d-1}$ to P_{rcv} , as well as their formats, are the same as if it had been playing honestly the protocol, the view of the adversary is unchanged.

We then define the cascade of $(n+1)(d-2)$ games:

$$(\mathbf{Game}^{r,i})_{r \in [d-2], i \in [n]} \quad (1)$$

for $i = 0, \dots, n$, then for $r = d-2, \dots, 1$ in backwards order. We set $\mathbf{Game}^{d-1,0} = \mathbf{Game}^{rcv}$ and $\mathbf{Game}^{r,0} = \mathbf{Game}^{r+1,n} \forall r = 2, \dots, d-1$. We then define recursively each $\mathbf{Game}^{r,i}$, for $i \geq 1$, as equal to $\mathbf{Game}^{r,i-1}$, with the change that P_i now behaves, as an old committee member, in the same way as simulated by Sim. First, since the modified checks pass if and only if the actual checks pass, the simulated honest parties send messages with the same volume as the ones sent in $\mathbf{Game}^{r,i}$. As regards the contents of the messages received by the corrupt parties, it follows from the t -privacy property of the Shamir Secret Sharing that they are equally distributed in $\mathbf{Game}^{r,i}$ as in $\mathbf{Game}^{r,i-1}$. In conclusion, the view of the adversary in $\mathbf{Game}^{r,i}$ is the same as in $\mathbf{Game}^{r,i-1}$.

We then define the last game **Game^{sdr}**, which is obtained from $\mathbf{Game}^{1,n}$ by making the simulated sender P_{sdr} follow the same behavior as the one simulated by Sim. Hence, we are now arrived at the simulated execution. Indistinguishability from $\mathbf{Game}^{1,n}$ again follows from the t -privacy property of the Shamir Secret Sharing.

□

C Broadcast Functionality and Construction

We define the functionality of $\mathcal{F}_{bc}$ below, which is also used in prior papers for MPC with dynamic committees.

Broadcast Channel $\mathcal{F}_{bc}$

Public Parameters. Senders $P_{sdr} \in \mathcal{L}_0$, receiver layer $\mathcal{L}_k$ where $k \geq 1$. The domain M of the message. Set $m = \perp$.

Layer $\mathcal{L}_0$:

- The functionality receives a message $x \in M$ from $P_{sdr} \in \mathcal{L}_0$ (where a corrupted sender can input the message in a rushing fashion, after the honest parties sent their messages of this round), towards all parties in $\mathcal{L}_k$. Set $m = x$.
- Output m to the adversary.

Layer $\mathcal{L}_k$:

- If the sender P_{sdr} is corrupted, receive m' from the (ideal) adversary. Set $m = m'$.
- Output m to all parties in $\mathcal{L}_k$.

C.1 Protocol Description.

Consider the straightforward protocol Π_{bc} that simply lets the $P_{\text{sdr}} \in \mathcal{L}_0$ send its message x to all parties in $\mathcal{L}_1$. Then, each $P_1 \in \mathcal{L}_1$ inputs x to a layered consensus protocol $\Pi_{\text{consensus}}$ taking $d = O(\lambda)$ layers (we choose the one in corollary 12). Finally, parties in $\mathcal{L}_{d+1}$ output what they receive from the consensus protocol.

Theorem 18. *There exist layered broadcast protocols with (1) statistical security for $t < n/3$ and no setup assumptions and (2) cryptographic security for $t < n/2$ assuming a plain PKI.*

Proof. (Sketch)

First note that the outputs of honest parties are the same in both real and ideal worlds. If the sender $P_{\text{sdr}} \in \mathcal{L}_0$ is honest with input x , the broadcast functionality outputs x in the ideal world. In the real world, all parties in layer $\mathcal{L}_1$ obtain the same input x , and by validity of $\Pi_{\text{consensus}}$, all honest parties output x . In case the sender $P_{\text{sdr}} \in \mathcal{L}_0$ is corrupted, $\Pi_{\text{consensus}}$ guarantees that all honest parties output the same value x' at the end of the protocol. The simulator simply emulates all the steps of the protocol Π_{bc} on behalf of the honest parties, obtains the output x' of $\Pi_{\text{consensus}}$ and inputs it to $\mathcal{F}_{\text{bc}}$. The functionality then guarantees that honest parties also obtain the output x' in the ideal world.

The simulation of the adversary's view is straightforward. Consider an honest sender $P_{\text{sdr}} \in \mathcal{L}_0$. The simulator obtains the message x and simulates the whole protocol Π_{bc} on behalf of the honest parties. For the case of a corrupted sender, the simulator again simulates the whole protocol Π_{bc} on behalf of the honest parties. Note that since there is no secret information, the simulator can exactly emulate the adversary's view in the real world.